Nafisa

_IoT_Intrusion_Detection(1)

Turnitin LLC

Document Details

Submission ID
trn:oid:::9717:217647786

Submission Date
Sep 8, 2026, 3:38 AM GMT+10

Download Date
Sep 8, 2026, 3:41 AM GMT+10

File Name
_IoT_Intrusion_Detection(1).pdf

File Size
2.2 MB

62 Pages

14,647 Words

84,908 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

1 AI-generated 0%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

ABSTRACT

The rapid adoption of IoT devices has expanded the network attack surface, making infrastructure increasingly vulnerable to DoS/DDoS attacks, backdoor attacks, injection attacks, ransomware, scanning attacks, password attacks, cross-site scripting (XSS), and man-in-the-middle attacks. Although machine learning and deep learning models are effective at intrusion detection, most operate as "black boxes" without providing information about why a network flow is classified as malicious. This thesis aims to develop an interpretable deep learning framework for IoT intrusion detection using the ToN-IoT dataset, considering both binary and ten-class attack-type classification. Two feature representations are compared: 23 principal components derived from 40 original features using Principal Component Analysis (PCA), and a novel Hybrid Ensemble Feature Selection approach combining filter, embedded, wrapper, and SHAP-based rank aggregation methods to retain 20 interpretable features. SMOTE is applied only to the training split to prevent data leakage. Seven classifiers such as Random Forest, KNN, XGBoost, CatBoost, TabTransformer, an Autoencoder Classifier, and a Temporal Convolutional Network—are benchmarked on both feature representations using 5-fold cross-validation. The best results are achieved by Random Forest using the Hybrid Feature Selection representation, attaining accuracy comparable to that of PCA for both binary (99.91%) and multiclass (98.93%) classification while maintaining feature interpretability. SHAP is then applied to the best-performing model to generate global and local explanations, identifying dominant attack indicators such as destination port, protocol, and DNS-related features. The results demonstrate that high predictive accuracy and model transparency are not mutually exclusive.

Keywords: Intrusion Detection System, IoT Security, ToN-IoT, Explainable AI, SHAP, Random Forest, TabTransformer, Feature Selection, PCA, Deep Learning.

TABLE OF CONTENTS

DECLARATION ..... Error! Bookmark not defined.
SUPERVISOR'S DECLARATION ..... Error! Bookmark not defined.
DECLARATION OF THESIS / PROJECT REPORT AND COPYRIGHT. Error! Bookmark not defined.
ACKNOWLEDGEMENT ..... Error! Bookmark not defined.
ABSTRACT ..... **i**
TABLE OF CONTENTS ..... **ii**
LIST OF TABLES ..... **iv**
LIST OF ILLUSTRATIONS ..... **v**
LIST OF ABBREVIATIONS ..... **vii**

CHAPTER I
INTRODUCTION **1**
1.1 Research Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objective of Research 2
1.5 Organization of the Thesis 3
CHAPTER II
LITERATURE REVIEW **4**
2.1 Introduction 4
2.2 Literature Review 4
2.2.1 General 4
2.2.2 Specific 5
2.3 RESEARCH GAP 7
2.4 SCOPE OF THE RESEARCH 9
2.5 Summary 9
CHAPTER III
RESEARCH METHODOLOGY **10**
3.1 Introduction 10
3.2 Dataset Description 10
3.2.1 Target Distribution and Correlation Structure 11
3.3 Data Preprocessing 13
3.4 Train-Test Split and Class-Imbalance Correction 14
3.5 Dimensionality Reduction 15

3.5.1 Principal Component Analysis (PCA) 15
3.5.2 Hybrid Ensemble Feature Selection (Contribution) 16
3.5.3 PCA vs. Hybrid Feature Selection -- Quick Benchmark 19
3.6 Classification Models 20
3.7 Cross-Validation Protocol 21
3.8 Explainable AI (SHAP) 22
3.9 Evaluation Metrics 23
3.10 Summary 24
CHAPTER IV
RESULTS AND DISCUSSION 25
4.1 Introduction 25
4.2 Results -- Binary Classification (Attack vs. Normal) 25
4.2.1 Per-Model Performance 25
4.2.2 5-Fold Cross-Validation 27
1.2.3 Final Model Comparison and Explainability 28
4.3 Results -- Multiclass Classification (10 Attack Types) 31
4.3.1 Per-Model Performance 31
4.3.2 5-Fold Cross-Validation 33
4.3.3 Final Model Comparison and Explainability 34
4.3.4 Statistical Significance Testing 38
4.3.5 Probability Calibration Analysis 42
4.4 Discussion 47
4.5 Comparison with State-of-the-Art (ToN-IoT) 48
4.6 Summary 49
CHAPTER V
CONCLUSION AND FUTURE WORKS 50
5.1 Conclusion 50
5.2 Contribution of this Thesis 50
5.3 Future Works 51
REFERENCES 53

LIST OF TABLES

Table No **Page**

Table 2.1: Literature review of ToN-IoT / IoT intrusion-detection studies (2022–2026) 5
Table 3.1: Class distribution of the ToN-IoT dataset used in this study 11
Table 3.2: Three ToN-IoT records before and after the preprocessing pipeline ... 14
Table 3.3: Top-10 principal components by explained variance (binary-task pipeline), with each component's dominant original feature 16
Table 3.4: Hybrid Ensemble Feature Selection -- top-10 features by average rank across MI, XGBoost importance, RFE, and SHAP (binary task) 19
Table 3.5: XGBoost probe accuracy: PCA vs. Hybrid Feature Selection 20
Table 4.1: Binary task -- accuracy, ROC-AUC and PR-AUC by model and feature representation 25
Table 4.2: Random Forest (Hybrid FS) -- binary classification report on the held-out test set 26
Table 4.3: Binary task -- 5-fold stratified cross-validation results (Hybrid FS features) 27
Table 4.4: Random Forest (Hybrid FS) -- top-15 features by mean |SHAP| value, binary task (positive class: Attack) 28
Table 4.5: Multiclass task -- accuracy by model and feature representation 31
Table 4.6: Random Forest (Hybrid FS) -- multiclass classification report on the held-out test set 32
Table 4.7: Multiclass task -- 5-fold stratified cross-validation results (Hybrid FS features) 33
Table 4.8: Random Forest (Hybrid FS) -- top-15 features by mean |SHAP| value averaged across all classes, multiclass task 35
Table 4.9: Corrected resampled paired t-test — Random Forest vs. each baseline (5-fold CV, multiclass, Hybrid FS) 39
Table 4.10: McNemar's test on the hold-out test set — Random Forest vs. each baseline (multiclass, Hybrid FS, n = 42,209) 40
Table 4.11: Consolidated statistical significance summary — Random Forest vs. each baseline, multiclass task 41
Table 4.12: Calibration metrics uncalibrated models, multiclass task (Hybrid FS, hold-out test set, n = 42,209, 15 bins) 42
Table 4.13: Post-hoc temperature-scaling recalibration — all models, multiclass task (evaluation split, n = 29,547) 44
Table 4.14: Selective-prediction operating points — Random Forest (temperature scaled), multiclass task 47
Table 4.9: Comparison with related ToN-IoT intrusion-detection studies 49

LIST OF ILLUSTRATIONS

Figure No **Page**

Figure 3.1: Methodology of this study. 10

Figure 3.2: Binary target distribution -- Normal vs. Attack (count and proportion) 12

Figure 3.3: Multiclass (attack-type) target distribution -- count and proportion across all ten classes..... 12

Figure 3.4: Pearson correlation heatmap of the top 15 numeric features 12

Figure 3.5: Binary task -- training-set class distribution before and after SMOTE (test set left untouched) 15

Figure 3.6: Multiclass task -- training-set class distribution before and after SMOTE, showing the minority mitm class rebalanced from 834 to 40,000 records 15

Figure 3.7: PCA scree plot (top 30 components) and cumulative explained variance, with the 95% threshold at 23 components..... 16

Figure 3.8: Working of the Hybrid Ensemble Feature Selection pipeline four orthogonal ranking branches fused by average rank into a top-20 interpretable feature set. 18

Figure 3.9: Hybrid Feature Selection -- top-20 features ranked by ensemble average rank (binary task)..... 19

Figure 3.10: PCA vs. Hybrid Feature Selection -- XGBoost probe accuracy, binary task 20

Figure 4.1: Confusion matrix -- Random Forest (Hybrid FS), binary task 26

Figure 4.2: 5-fold stratified cross-validation -- mean accuracy ± std. dev. by model, binary task (Hybrid FS features)..... 27

Figure 4.3: Model comparison -- PCA vs. Hybrid Feature Selection across all seven models, binary task 28

Figure 4.4: Global SHAP feature importance (mean |SHAP| bar chart) -- Random Forest (Hybrid FS), binary task 29

Figure 4.5: Global SHAP beeswarm summary -- direction and magnitude of each feature's effect, binary task..... 29

Figure 4.6: Local SHAP waterfall explanation for one test sample predicted as Attack, binary task 30

Figure 4.7: Local SHAP waterfall explanation for one test sample predicted as Normal, binary task..... 31

Figure 4.8: Confusion matrix -- Random Forest (Hybrid FS), multiclass task. 33

Figure 4.9: 5-fold stratified cross-validation -- mean accuracy ± std. dev. by model, multiclass task (Hybrid FS features)..... 34

Figure 4.10: Model comparison -- PCA vs. Hybrid Feature Selection across all seven models, multiclass task 35

Figure 4.11: Global SHAP feature importance (mean |SHAP| bar chart, averaged across classes) -- Random Forest (Hybrid FS), multiclass task 36
Figure 4.12: Global SHAP beeswarm summary with respect to each sample's predicted class, multiclass task 36
Figure 4.13: Local SHAP waterfall explanation for one test sample, w.r.t. its predicted class (multiclass task) 37
Figure 4.14: Local SHAP waterfall explanation for a second test sample, w.r.t. its predicted class (multiclass task) 38
Figure 4.15: Critical difference diagram — Nemenyi post-hoc test over the 5-fold average ranks of all seven models, multiclass task 39
Figure 4.16: Paired bootstrap 95% confidence intervals for the accuracy difference between Random Forest and each baseline, multiclass task 41
Figure 4.17: Reliability diagrams — all seven uncalibrated models, multiclass task (Hybrid FS, hold-out test set) 43
Figure 4.18: Reliability diagram before vs. after recalibration — Random Forest, multiclass task 45
Figure 4.19: Per-class (one-vs-rest) calibration error before and after temperature scaling — Random Forest, multiclass task 46
Figure 4.20: Risk–coverage (selective prediction) curve — Random Forest, multiclass task, uncalibrated vs. temperature-scaled vs. isotonic 46

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CV	Cross-Validation
DDoS	Distributed Denial-of-Service
DL	Deep Learning
DNS	Domain Name System
DoS	Denial-of-Service
FS	Feature Selection
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
IIUC	International Islamic University Chittagong
IoT	Internet of Things
KNN	K-Nearest Neighbors
MI	Mutual Information
MITM	Man-in-the-Middle
ML	Machine Learning
PCA	Principal Component Analysis
PR-AUC	Precision-Recall Area Under Curve
RFE	Recursive Feature Elimination
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TCN	Temporal Convolutional Network
ToN-IoT	Telemetry of Network IoT (dataset name)
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting
XSS	Cross-Site Scripting

CHAPTER I

INTRODUCTION

1.1 RESEARCH BACKGROUND

The Internet of Things (IoT) is a groundbreaking technology that has emerged from the realm of possibility to become an essential element in today's technology landscape. Conceived as a vision of an interconnected web of billions of sensors, actuators and embedded devices across smart homes, industrial systems, healthcare and critical infrastructure, IoT is now a reality that is transforming the field of computing and innovation as we know it. But all this growth has not been accompanied by a similar level of security at the device level. Most of the IoT endpoints are resource-starved, have cramped firmware and often are used with default credentials or unpatched software, making them an enticing and relatively easy target for would-be attackers. One of the main defenses against these attacks is to use a Network Intrusion Detection System (IDS), which can be used to detect traffic patterns similar to malicious ones or search for signatures.

Novel or evolving attacks, which are difficult to be covered by traditional signature-based IDS, have pushed a gradual transition from IDS based on signature to ML and DL based IDS. Such models as Random Forest, Gradient Boosted Trees and more and more sophisticated tabular deep networks (transformers, autoencoders, temporal convolutional networks) are able to learn complex, non-linear decision boundaries directly from network-flow features and have been shown to achieve assessment and detection accuracy above 95% on public IoT benchmarks like ToN-IoT created by the Cyber Range and IoT Labs at UNSW Canberra (Moustafa, 2021; Booth, Chowdhury, & Barua, 2021). But there's a catch, and indeed the most accurate are often the least transparent – providing no principled understanding for security analysts on what network characteristics lead to such an “attack” verdict.

1.2 PROBLEM STATEMENT

A black-box IDS will result in a trust gap in a production security operations environment. Without support from the root traffic attributes, an analyst seeing one of

these labels (Attack/DDoS) - without knowing the underlying traffic sources - cannot easily verify the alert, prioritize a response, differentiate a true intrusion from a false positive, or argue to escalate to the incident response team. This problem is further exacerbated in the case of IoT traffic, which has a massive feature space and also a very-high dimensional space of attack (S) with multiple fields that are categorical, as well as numeric, and are imbalanced both within each attack class and across the entire space. Dimensionality reduction such as Principal Component Analysis (PCA) is a typical pre-processing step before dimensionality reduction that further diminishes the interpretability of the results — each Principal Component (PC) is a secure-less, linear combination of many of the original features. It is therefore clear that there is an urgent need for an intrusion detection pipeline that is (i) accurate, and can achieve high binary attack detection and fine-grained attack-type classification accuracy, (ii) robust to the severe class imbalance problem, and (iii) explainable both globally (model-wide) and locally (per-prediction).

1.3 MOTIVATION

We have seen some developments in ExAI (XAI) techniques recently, and specifically SHAP (SHapley Additive exPlanations) into practically usable tools that explain a model's prediction and can be comparatively fairly done with game-theoretic guarantees. Combining SHAP with an IoT IDS pipeline can provide a way to create systems that can be accurate and auditable: Taking a global look at all SHAP summaries can yield insights into the most important network characteristics throughout all traffic, while looking at the waterfall summary for a single flow can reveal why a particular flow is flagged. Another reason for this work is methodological; rather than only use PCA which loses the identity of the features, this thesis develops and tests a Hybrid Ensemble Feature Selection strategy consisting of four independent ranking methods, namely Mutual Information, embedded XGBoost importance, wrapper based Recursive Feature Elimination, and SHAP based ranking, which resulted in a set of interpretable, meaningful and well-named features while maintaining, or improving, downstream accuracy relative to PCA.

1.4 OBJECTIVE OF RESEARCH

The specific objectives of this research are:

1. To design a complete preprocessing and feature-engineering pipeline for the ToN-IoT network-flow dataset, including missing-value handling, categorical encoding, class-imbalance correction (SMOTE applied strictly to the training split), and two parallel dimensionality-reduction strategies (PCA and Hybrid Ensemble Feature Selection).

2. To train and rigorously benchmark seven classifiers -- Random Forest, K-Nearest Neighbors, XGBoost, CatBoost, TabTransformer, an Autoencoder-based Classifier, and a Temporal Convolutional Network (TCN) -- on both binary (attack vs. normal) and ten-class (attack-type) classification tasks.

3. To validate model performance using 5-fold stratified cross-validation and standard metrics (accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC) alongside confusion-matrix analysis.

4. To apply SHAP-based explainability to the best-performing model in each task, producing global feature-importance rankings and local, instance-level explanations for individual predictions.

5. To compare the PCA-based and Hybrid Feature-Selection-based pipelines directly, quantifying the trade-off (if any) between interpretability and predictive accuracy.

1.5 ORGANIZATION OF THE THESIS

The rest of this thesis will be structured as follows. Chapter II provides a review of existing related work including the work that has been done on machine-learning intrusion detection in IoT networks, as well as the increasing number of research work that focused about the explainable AI in cybersecurity, and the gap that this thesis is filling. The research methodology is presented in detail in Chapter III, including the dataset, preprocessing pipeline, feature-reduction approaches, seven classification models, evaluation protocol and the SHAP-based explainability approach. The experimental results are presented and discussed in Chapter IV along with various comparative tables, confusion matrices, results from cross-validation and SHAP visualization plots. The thesis' summary and future work directions are presented in Chapter V which concludes the thesis.

CHAPTER II

LITERATURE REVIEW

2.1 INTRODUCTION

In this chapter, the existing research is examined with the objective of highlighting the relevance of the current research for the study while covering three related topics: (i) machine learning and AI based intrusion detection systems for networks, (ii) feature selection and dimensionality reduction techniques on high dimensional network-flow data, and (iii) the use of explainable AI especially SHAP in the field of cyber security applications. Finally the chapter concludes by positioning the current project against this literature and stating the identifying gap in this field.

2.2 LITERATURE REVIEW

2.2.1 General

Intrusion detection research has come through multiple waves of methodology, from rule- and signature-based approaches, through classical statistical-based anomaly detection, the so-called “shallow” machine learning methods (decision tree, support vector machines, k-nearest neighbors), and the recent deep learning architectures that learn hierarchical representations directly from unprocessed or only lightly engineered network-flow features. Ensemble methods such as Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016), and CatBoost (Dorogush, Ershov, & Gulin, 2018) perform very well on tabular network data, are fast to train, handle mixed categorical/numeric features naturally, and are relatively noise resistant; deep tabular methods such as TabTransformer (Huang, Khetan, Cvitkovic, & Karnin, 2020), which borrows the attention mechanism originally proposed by Vaswani et al. (2017), attempt to bring the representational power of self-attention to tabular datasets. A related, parallel story is dimensionality reduction: raw network-flow captures may contain dozens of correlated statistical features, and Principal Component Analysis (PCA) is commonly employed to reduce the feature space, but at the expense of losing the original meaning of the features — a price that becomes very costly once explainability is required. Where class imbalance is severe, as is typical of rare attack types in IoT traffic, resampling techniques such as SMOTE (Chawla, Bowyer, Hall, & Kegelmeyer,

2002) are commonly applied on the training partition to avoid biasing the classifier toward the majority class.

2.2.2 Specific

Table 2.1 provides an overview of 20 studies published between 2022 and 2026 that fall within the scope of ToN-IoT (Moustafa, 2021; Booth, Chowdhury, & Barua, 2021) and related intrusion-detection research. These studies cover four types of feature-selection strategy (filter, wrapper, embedded, and metaheuristic/hybrid), two kinds of classifier (classical and deep), and differing extents of explainability.

Table 2.1: Literature review of ToN-IoT / IoT intrusion-detection studies (2022–2026)

Author(s) / Year	Method	Significance	Limitation
Alani & Miri (2022)	Universal 6-feature filter-based selection + SHAP, evaluated on ToN-IoT, IoT-23, IoT-ID	99.62% accuracy with a compact, cross-dataset universal feature set; SHAP validates feature relevance against attack behavior	Validated only on a Decision Tree classifier; not benchmarked against deep tabular architectures
Kareem, Mostafa, Hashim & El-Bakry (2022)	Hybrid Gorilla Troops Optimizer + Bird Swarm Algorithm (GTO-BSA) metaheuristic feature selection	Shows metaheuristic FS can reduce dimensionality while preserving accuracy across IoT benchmarks	Population-based search adds significant computational overhead; no XAI applied
Disha & Waheed (2022)	Gini Impurity-based Weighted Random Forest (GIWRF) embedded feature selection on ToN-IoT and UNSW-NB15	Embedded FS handles class imbalance directly via weighted splitting; reduces to 10 features on ToN-IoT	Binary classification only; no explainability layer
Nazir, Memon, Sadiq, Rahman & Khan (2023)	Cellular Automata + Tabu Search (CAT-S) hybrid wrapper FS + Random Forest, ToN-IoT	Improves accuracy and lowers false-positive rate versus prior wrapper approaches with fewer features	No XAI applied; wrapper search adds training-time overhead
Mahalingam et al. (2023)	ROAST-IoT: Scattered Range Feature Selection + attention-based convolutional network, ToN-IoT 99.78%	Attention mechanism with range-based FS achieves state-of-the-art accuracy across four IoT datasets	Deep attention architecture is not interpretable; no per-prediction explanation
Njeri et al. (2024)	CNN-LSTM feature extraction + SVM/RF classification, ToN-IoT	Combines deep feature extraction with classical classifiers; strong multiclass accuracy	No explainability (XAI) component; feature importance not reported
Maseno, Mwandata, Wang & Sun (2024, CSAI)	CNN-LSTM hybrid feature reduction + SVM/RF, SMOTETomek balancing, ToN-IoT (RF 98%, SVM 91%)	Hybrid deep-feature-reduction pipeline improves classical classifier accuracy	Feature-reduction stage is not interpretable; no per-prediction explanation
Zhang et al. (2024)	AOC-IDS: autoencoder + contrastive learning	Contrastive objective improves separability of minority attack classes	Black-box representation; no post-

			hoc explanation of predictions
Bamber, Katkuri, Sharma & Angurala (2025)	Hybrid CNN-LSTM architecture for IoT intrusion detection	Captures both spatial and temporal patterns in traffic sequences	No XAI applied; limited discussion of class-imbalance handling
Rhachi et al. (2025)	Deep autoencoder feature learning + ANOVA-based feature selection	Unsupervised representation learning reduces manual feature engineering	Autoencoder latent features are not human-interpretable; no XAI applied
Logeswari, Thangaramya, Selvi & Roselind (2025)	Synergistic Dual-Layer FS (SDFC): mutual information + variance thresholding + SVM-RFE + PSO, ToN-IoT	Combines statistical and model-based FS layers to improve feature relevance for IoT anomaly detection	High architectural complexity; no SHAP/LIME-based validation of selected features
Abou Elasaad, Sayed & El-Dakroury (2025)	AegisGuard: quantum-inspired hybrid FS + RF/Extra Trees/LightGBM/XGBoost/ CatBoost ensemble, ToN-IoT 98.3%	Trust-aware weighted FS with Optuna-tuned ensemble cuts feature dimensionality by 54% while reducing false alarms	Quantum-inspired weighting adds design complexity; no per-prediction explainability
Shaikhanova, Kuznetsov, Tokkuliyeva, Ayapbergenov, Olzhas & Danir (2025)	RF/LightGBM/XGBoost/Logistic Regression benchmarking with computational profiling, ToN-IoT (99.8–99.9% accuracy)	Reports full computational benchmarks (inference time, model footprint) alongside accuracy for real-time deployment	Explicit 78% F1-score blind spot on MITM despite 99% overall accuracy; no XAI to diagnose the gap
Mustafa, Eljack Babiker & Mustafa (2025)	Hybrid Recurrent + Spiking Neural Network (HRSNN) with SMOTE and Recursive Feature Elimination	Combines temporal RNN feature extraction with energy-efficient spiking classification	Spiking-network black-box behavior; no explainability component
Zahoor, Abbasi, Babar & Aljohani (2025)	Isolation Forest + One-Class SVM unsupervised anomaly detection with LIME, ToN-IoT	Avoids labeled-data dependency; LIME adds local interpretability; tested under adversarial label-flip poisoning	Unsupervised anomaly scores under-perform supervised methods on fine-grained multiclass attack typing
Wakili, Guni, Khan, Yu & Jung (2025)	Unified benchmarking of classical, deep, and hybrid IoT/IIoT models on power-consumption side-channel data	Cross-device benchmarking highlights accuracy-efficiency trade-offs across model families	Uses power side-channel rather than network-flow features; not evaluated on ToN-IoT directly
Tawfik (2024)	Stacked autoencoders + CatBoost + optimized Transformer-CNN-LSTM ensemble for fog/IoT intrusion detection	Ensemble targets resource-constrained fog nodes, combining representation learning with gradient boosting	No dataset-bias analysis; no explainability component reported
Almalawi (2025)	Recursive clustering data condensation + Monte Carlo Cross-Entropy feature-stability selection for lightweight IoT IDS	Reduces training/testing time by up to 45×/15× while maintaining comparable accuracy across three IoT datasets	Not evaluated on ToN-IoT; no explainability layer
Dharini, Janani & Katiravan (2026)	Bias-aware hybrid FS (5 features): domain filtering + forward/backward/genetic wrapper selection, ToN-IoT	Explicitly removes IP/port bias; achieves linear time complexity O(n·k) suitable for edge deployment	Aggressive 5-feature reduction lowers MITM-class precision; no SHAP-based validation of selected features

Hossain, Khan, Islam, Islam & Gedeon (2026)	Cross-dataset generalization study: RF/Logistic Regression/Naive Bayes on ToN-IoT and UNSW-NB15	RF reaches 99.79% same-dataset accuracy on ToN-IoT, exposing a severe cross-dataset generalization gap (<40%)	No feature-selection or explainability contribution; highlights an unsolved generalization problem rather than a fix

The theme common to all of these studies is that those seeking the best raw accuracy tend to rely on more complex feature-extraction pipelines that are often opaque (autoencoders, CNN-LSTM hybrids, spiking networks), as in Zhang et al. (2024), Rhachi et al. (2025), and Mustafa, Eljack Babiker, and Mustafa (2025), while those seeking explainability tend to employ simpler feature sets, which can sacrifice discriminative power — particularly for rare attack classes such as MITM, as reported by Shaikhanova et al. (2025) and Dharini, Janani, and Katiravan (2026). No study in Table 2.1 applies SHAP consistently throughout the whole pipeline, both at the feature-selection stage and at the model-explanation stage, and no study directly compares a PCA-based pipeline against a fully named, interpretable hybrid-feature-selection pipeline on the same dataset and the same set of models.

The wider XAI-for-cybersecurity literature has generated considerable interest in SHAP, since it rests on a well-founded basis in cooperative game theory (Lundberg & Lee, 2017) and has emerged as the pre-eminent post-hoc explanation method for tabular intrusion-detection models because of its capability to generate both global feature-importance rankings (mean absolute SHAP value) and local, additive, per-instance explanations (via waterfall and force plots). TreeExplainer (Lundberg & Lee, 2017) delivers efficient and exact SHAP computation for tree ensembles (Random Forest, XGBoost, CatBoost), while KernelExplainer (Lundberg & Lee, 2017) is slower but model-agnostic, allowing SHAP values to be computed for deep or distance-based models such as KNN, TabTransformer, autoencoder-based classifiers, and temporal convolutional networks (Oord et al., 2016).

2.3 RESEARCH GAP

The literature reviewed in Section 2.2 exposes a consistent trade-off rather than a single missing technique: studies either push raw accuracy through opaque, complex feature-extraction and classification pipelines, or they preserve interpretability by trimming down to a small, simple feature set at the cost of discriminative power on rare attack

types. From this pattern, four specific gaps can be identified that this thesis sets out to address.

i. No end-to-end, consistent application of SHAP. Several studies apply SHAP either only at the feature-selection stage (Alani & Miri, 2022) or only for post-hoc model explanation, but none of the reviewed works apply a single, consistent SHAP-based rationale across both the feature-selection stage and the final model-explanation stage of the same pipeline.

ii. No direct, like-for-like comparison between PCA and a named, interpretable hybrid feature-selection pipeline. Dimensionality-reduction approaches (PCA) and interpretable hybrid feature-selection approaches (filter, wrapper, embedded, and SHAP rank-aggregation) have not been benchmarked side by side on the same dataset, the same train/test protocol, and the same set of classifiers, so it remains unclear how much accuracy — if any is actually sacrificed for interpretability.

iii. Unresolved accuracy–interpretability trade-off on rare attack classes. Studies that aggressively reduce the feature set for interpretability, such as the 5-feature selection of Dharini, Janani, and Katiravan (2026), report a measurable drop in precision on minority classes such as MITM, while studies that preserve accuracy on such classes, such as Shaikhanova et al. (2025), do so without any explainability mechanism to diagnose why the model struggles. No reviewed study evaluates whether a hybrid ensemble feature-selection approach (filter + embedded + wrapper + SHAP rank aggregation) can retain both minority-class performance and interpretability simultaneously.

iv. Cross-dataset generalization remains unaddressed by feature-selection or XAI work. Hossain, Khan, Islam, Islam, and Gedeon (2026) expose a severe cross-dataset generalization gap for ToN-IoT-trained models, but their study contributes neither a feature-selection strategy nor an explainability layer that could help diagnose or mitigate this gap; no reviewed feature-selection or XAI study engages with the generalization problem at all.

Taken together, these four gaps define the specific niche this thesis occupies: an interpretable deep-learning framework for IoT intrusion detection, evaluated on ToN-IoT in both binary and ten-class settings, that (a) uses a novel Hybrid Ensemble Feature

Selection combining filter, embedded, wrapper, and SHAP rank-aggregation to retain a fully named, interpretable feature set; (b) benchmarks this representation directly against a PCA-reduced representation across seven classifiers Random Forest, KNN, XGBoost, CatBoost, TabTransformer, an Autoencoder Classifier, and a Temporal Convolutional Network under identical 5-fold cross-validation; (c) applies SMOTE only on the training split to avoid leakage; and (d) applies SHAP consistently, both to guide feature selection and to generate global and local explanations of the best-performing model, so that the dominant attack indicators can be identified without sacrificing detection accuracy.

2.4 SCOPE OF THE RESEARCH

This review focused on studies where (a) the ToN-IoT dataset or closely related IoT/network intrusion datasets are used, (b) a machine-learning architecture for intrusion or threat detection is proposed that is hybrid or ensemble in nature, and (c) an explainability mechanism is incorporated, or explicitly not considered. This scoping permits a straightforward like-for-like comparison between the framework proposed in this thesis and the prior work that is most similar to it.

2.5 SUMMARY

This chapter summarized the development of IoT IDS from signature-based towards category based and ensemble/deep-learning classifiers, reviewed the top five IoT IDS studies that were directly comparable, and emphasized the gap that an integrated pipeline has not been established that not only delivers competitive multiclass accuracy, but it also offers a fully named and interpretable feature-selection stage as well as a consistent SHAP-based explanation of the entire process and the final model. To fill that gap the methodology developed in this thesis is presented in Chapter III.

CHAPTER III

RESEARCH METHODOLOGY

3.1 INTRODUCTION

In this chapter, the data set is described, followed by the data pre-processing and exploratory data analysis (EDA) process, the two dimensionality-reduction processes (PCA and Hybrid Ensemble Feature Selection), the seven classification algorithms (which were used for benchmarking), the evaluation protocol, (single train/test split plus 5-fold stratified cross validation), and the process of applying explainability measures to the top-performing model (SHAP). The same pipeline is used twice, once with a binary target (Normal vs. Attack), and the second time with the original ten-class attack-type target, so that the detector is a coarse-grained version, and the classifier is a fine-grained version, of the same methodology.

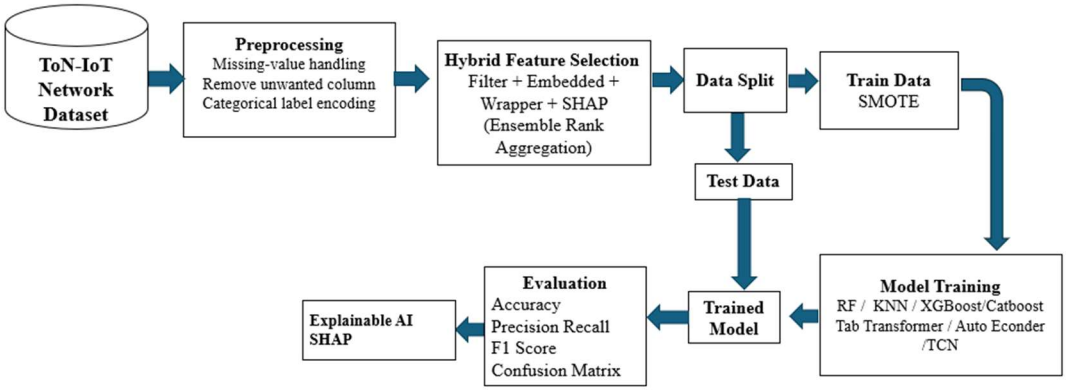


Figure 3.1: Methodology of this study.

3.2 DATASET DESCRIPTION

The study utilizes the ToN-IoT "train_test_network.csv" dataset, created by the Cyber Range and IoT Labs at UNSW Canberra (Moustafa, 2021; Booth, Chowdhury, & Barua, 2021), which features realistic IoT/IIoT network-flow telemetry data generated within a testbed consisting of benign and attack traffic. The raw dataset consists of 44 attributes with 211,043 flow records, including connection attributes (source/destination IP and port, protocol, service, duration, byte and packet counts, connection state), DNS attributes, SSL/TLS attributes, HTTP attributes, and Zeek anomaly indicators (weird), as well as two label columns: a binary label (0 = normal, 1 = attack) and a fine-grained

type column identifying the type of attack. There are 10 classes in the type column: normal traffic and the nine types of attacks: backdoor, DDoS, DoS, injection, password, ransomware, scanning, cross-site scripting (XSS), man-in-the-middle (MITM).

The columns src_ip and dst_ip are removed before modeling because the raw IP address of the origin of an attack would not generalize and it would be much easier to memorize attack signatures than learn them. Because the type column would trivially expose the multiclass attack category -- and thus the binary label -- straight into the feature set, it is dropped from the feature matrix along with the binary classification target, the binary label. This leaves 40 candidate features for modelling both tasks.

Table 3.1: Class distribution of the ToN-IoT dataset used in this study

Class	Binary Label	Record Count	Share of Total
normal	Normal	50,000	23.7%
backdoor	Attack	20,000	9.5%
ddos	Attack	20,000	9.5%
dos	Attack	20,000	9.5%
injection	Attack	20,000	9.5%
password	Attack	20,000	9.5%
scanning	Attack	20,000	9.5%
ransomware	Attack	20,000	9.5%
xss	Attack	20,000	9.5%
mitm	Attack	1,043	0.5%

The data set is somewhat imbalanced at the binary level (161,043 Attack versus 50,000 Normal records, a ≈ 3.2:1). The imbalance at the 10 class level is extreme, with the mitm class having only 1043 records compared to over 20,000 records for most other attack classes and more than 50,000 records for normal traffic, that is, a ratio of approximately 48:1 with the largest attack class. This imbalance directly propels the needed SMOTE oversampling step, as described in Section 3.5.

3.2.1 Target Distribution and Correlation Structure

Figure 3.2 and Figure 3.3 show the resulting class distributions for the binary and multiclass targets respectively, while Figure 3.4 presents the Pearson correlation structure among the fifteen numeric features most strongly correlated with the target, providing an initial view of redundancy among the raw features ahead of the dimensionality-reduction stage in Section 3.5

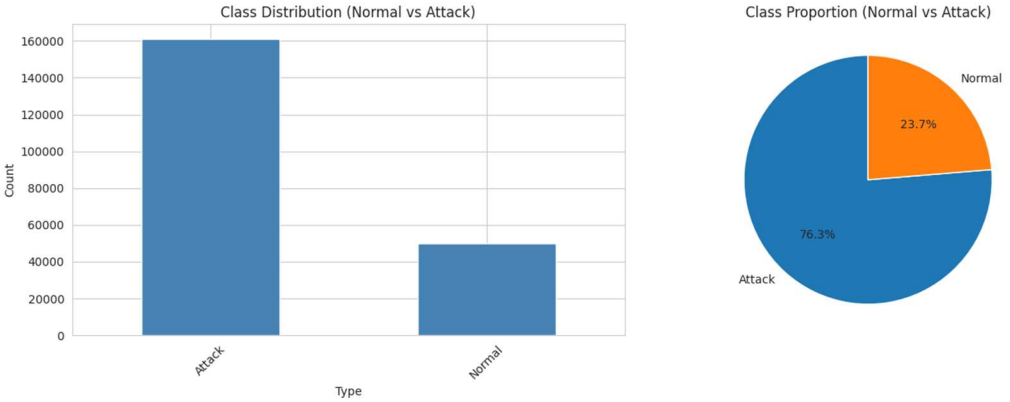


Figure 3.2: Binary target distribution -- Normal vs. Attack (count and proportion)

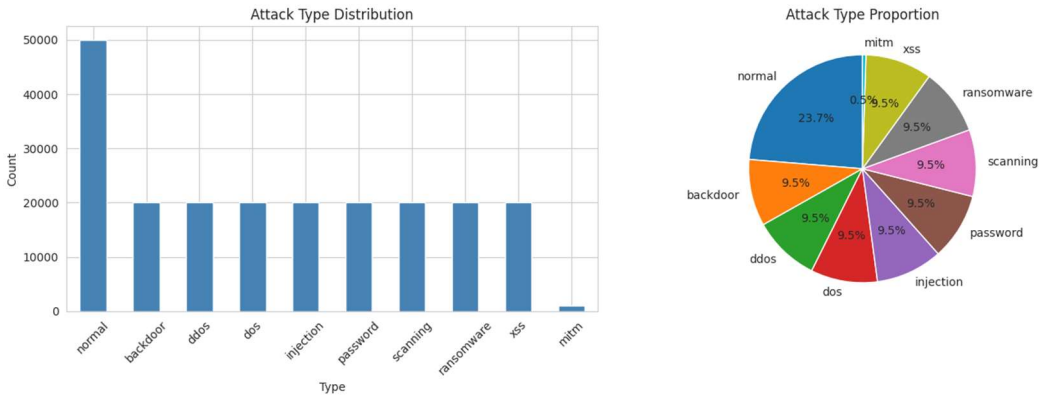


Figure 3.3: Multiclass (attack-type) target distribution -- count and proportion across all ten classes

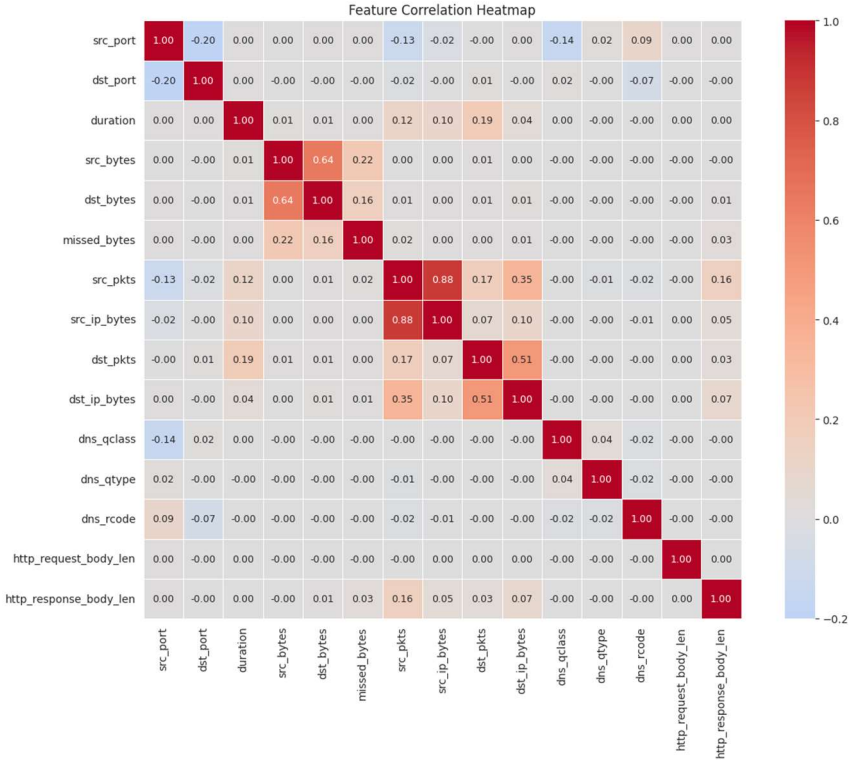


Figure 3.4: Pearson correlation heatmap of the top 15 numeric features

3.3 DATA PREPROCESSING

The preprocessing pipeline, applied identically to both the binary and multiclass tasks, consists of five steps. Each step is described below together with a short, concrete example drawn from the ToN-IoT feature set to illustrate exactly how a raw record is transformed before it reaches the dimensionality-reduction stage.

1. **Placeholder replacement.** The literal "-" character used throughout the raw ToN-IoT export to denote a missing or not-applicable value is replaced with NaN. For example, a plain TCP connection with no associated DNS activity is exported with dns_query = "-" and ssl_version = "-"; both are converted to NaN before any further processing so that they are not mistaken for a legitimate categorical level such as the string "0".

2. **Identifier removal.** src_ip and dst_ip are dropped. A raw record such as src_ip = 192.168.1.193, dst_ip = 192.168.1.1 is removed at this step rather than encoded, since a specific testbed address (e.g., 192.168.1.1, the gateway used to generate the attack traffic) would let a classifier memorize which host is attacking rather than learn the traffic pattern that makes a flow malicious.

3. **Target separation.** For the binary task, label is mapped to {0: "Normal", 1: "Attack"} and both label and type are removed from the feature matrix; for the multiclass task, type (e.g., "ddos", "password", "mitm") is retained as the target and label is removed instead. This prevents either target from leaking into the feature matrix of the other task.

4. **Missing-value imputation.** Numeric columns are filled with 0 (for instance, a UDP flow with no application-layer response leaves http_trans_depth blank, which becomes 0); categorical columns are filled with the string "missing" (for instance, a flow with no HTTP component leaves http_method blank, which becomes the literal category "missing" rather than being dropped, so the absence of HTTP activity is itself preserved as a signal).

5. **Categorical encoding.** All 24 categorical columns (proto, service, conn_state, and the DNS/SSL/HTTP flag fields) are label-encoded. For example, the three-level proto column {tcp, udp, icmp} is mapped to the integers {2, 1, 0}

respectively, and the conn_state column {S0, SF, REJ, RSTO, …} is mapped to a corresponding set of small integers in the same way; the target is separately label-encoded with scikit-learn's LabelEncoder (for the multiclass task, ddos → 1, mitm → 4, normal → 5, and so on), and the resulting class_names array is retained so that later reports and confusion matrices can be printed with the original class names rather than integer codes.

Table 3.2 walks through this pipeline on three illustrative records so that the net effect of all five steps can be seen on real values side by side, before and after preprocessing.

Table 3.2: Three ToN-IoT records before and after the preprocessing pipeline

Record	Raw proto	Raw dns_query	Encoded proto	Imputed value	Target
#1 (normal)	tcp	google.com	2	google.com	0 / normal
#2 (scanning)	tcp	"-"→NaN	2	missing	1 / scanning
#3 (mitm)	udp	"-"→NaN	1	missing	1 / mitm

After preprocessing, the feature matrix contains 211,043 rows × 40 columns (16 numeric, 24 encoded categorical), with zero remaining missing values. This clean, fully numeric matrix is the common starting point for both dimensionality-reduction paths described in Section 3.5.

3.4 TRAIN-TEST SPLIT AND CLASS-IMBALANCE CORRECTION

The data set is split 80:20 using a stratified sampler (random_state = 42) to preserve the proportions of each class in both the training and held-out test sets: (168,834, 42,209) for Task 1 and (372,313… 101,212) samples–feature pairs for Task 2 respectively (train, test). The train-test split is performed once, and the Synthetic Minority Over-sampling Technique (SMOTE, k = 5 neighbors) is applied only to the training partition to compensate for the extreme class imbalance found in Section 3.2. This ordering was made on purpose: in some previous pipelines, this step is performed in reverse — SMOTE before splitting — which allows synthetic samples generated from test-set neighbors to leak into the training set, leading to an over-optimistic accuracy report. In this pipeline, the test set is strictly preserved and left untouched, while only the training set is rebalanced: for the binary task, the training-set class distribution goes from 128,834 Attack / 40,000 Normal to 128,834 Attack / 128,834 Normal; for the multiclass

task, the training-set class distribution becomes 40,000 records for every class (with the minority mitm class upsampled from 834 to 40,000), while the test set remains unchanged. All features are subsequently standardized (zero mean, unit variance) using a StandardScaler fit only on the training set. Figure 3.5 and Figure 3.6 illustrate the resulting training-set class distributions before and after SMOTE for the binary and multiclass tasks respectively, confirming that the test set (not shown) is unaffected by the resampling step.

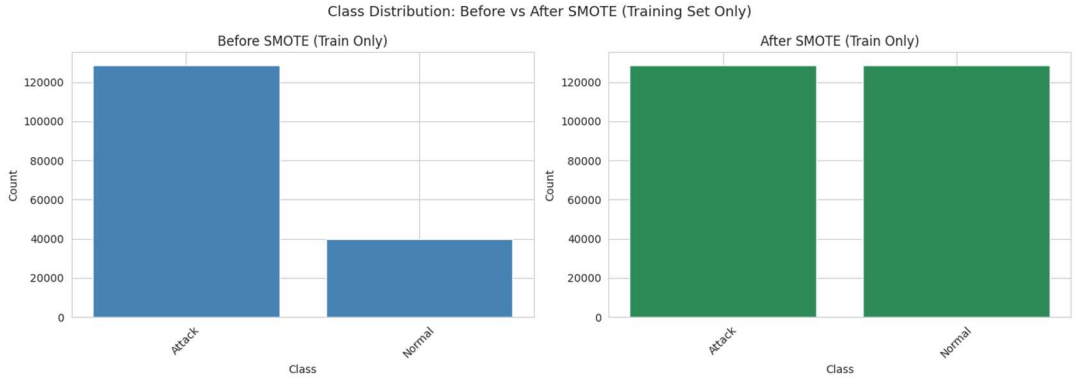


Figure 3.5: Binary task -- training-set class distribution before and after SMOTE (test set left untouched)

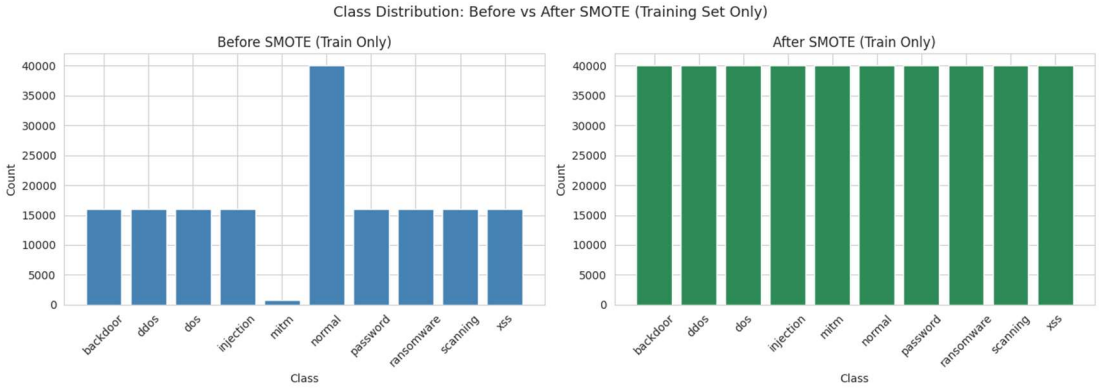


Figure 3.6: Multiclass task -- training-set class distribution before and after SMOTE, showing the minority mitm class rebalanced from 834 to 40,000 records

3.5 DIMENSIONALITY REDUCTION

3.5.1 Principal Component Analysis (PCA)

PCA is used as a baseline dimensionality-reduction technique on the standardized 40-feature training set. The number of components is chosen as the smallest number that explains at least 95% of total variance, which in both the binary and multiclass pipelines is 23 components a 42.5% reduction in dimensionality, as shown by the scree plot in

Figure 3.7. Table 3.3 lists the top-ranking components together with the original feature each one loads most heavily on.

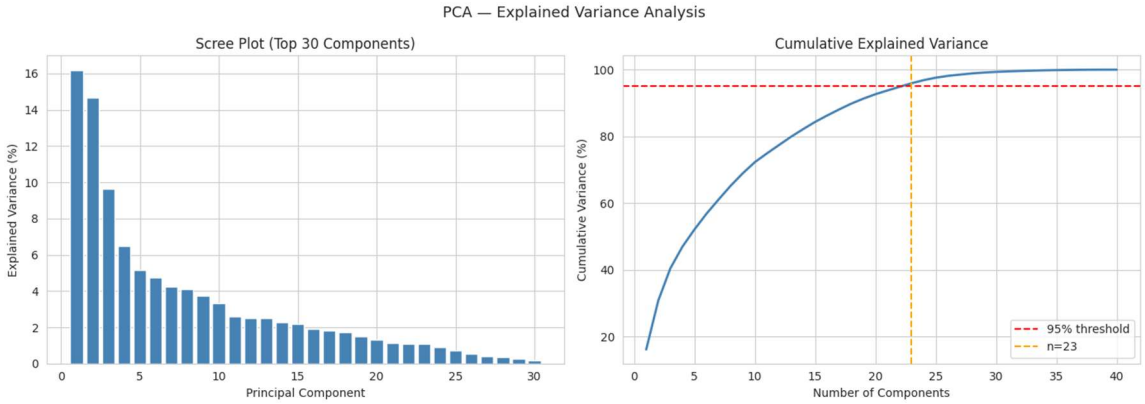


Figure 3.7: PCA scree plot (top 30 components) and cumulative explained variance, with the 95% threshold at 23 components.

As can be seen, while PCA is effective for compression, each retained component is only a linear combination of the original features and is therefore not directly interpretable as a specific, named network attribute.

Table 3.3: Top-10 principal components by explained variance (binary-task pipeline), with each component's dominant original feature

Rank	Component	Top Contributing Feature	Explained Variance (%)	Cumulative (%)
1	PC1	dns_RA	16.18	16.18
2	PC2	http_version	14.66	30.84
3	PC3	ssl_resumed	9.65	40.49
4	PC4	weird_name	6.48	46.97
5	PC5	src_pkts	5.17	52.14
6	PC6	ssl_subject	4.73	56.87
7	PC7	src_bytes	4.23	61.10
8	PC8	conn_state	4.12	65.23
9	PC9	http_orig_mime_types	3.72	68.95
10	PC10	dst_pkts	3.34	72.28

3.5.2 Hybrid Ensemble Feature Selection (Contribution)

PCA reduces correlated features to synthetic components and, in doing so, loses the identity of each original feature along with its direct interpretability. As the central methodological contribution of this thesis, a Hybrid Ensemble Feature Selection pipeline is developed that instead combines four independent, orthogonal ranking

strategies through average-rank fusion, so that the final selected feature set is composed entirely of real, named network attributes rather than synthetic projections.

1. **Filter — Mutual Information (MI).** The statistical dependency between each feature and the target is measured directly from the data, independent of any downstream model, so this branch is fast and captures simple, possibly non-linear, univariate relevance.
2. **Embedded — XGBoost gain-based importance.** A gradient-boosted tree ensemble is fitted on the full 40-feature training set, and the total gain each feature contributes across all splits is used as its importance score, so this branch captures how useful a feature actually is once combined with others inside a real model.
3. **Wrapper — Recursive Feature Elimination (RFE).** Using an XGBoost estimator, features are removed iteratively based on their contribution to model performance, so this branch captures each feature's marginal contribution once weaker features have already been removed.
4. **Explainable — mean absolute SHAP value.** Computed with TreeExplainer on the same fitted XGBoost model, this branch ranks features by how much they actually move individual predictions away from the model's base rate, tying the feature-selection stage directly to the explanation stage used later in Section 3.8.

Each of the four ranking methods has its own known bias MI ignores feature interactions, embedded importance can overestimate high-cardinality features, and RFE is sensitive to the choice of estimator so the final ranking for a feature f is computed as the simple average of its rank under each of the four methods, as given in Equation (3.1):

$$AverageRank(f) = 1/4 \sum_{i \in \{MI,XGB,RFE,SHAP\}} rank_i(f) \quad (3.1)$$

This works as follows in this study: all four ranking methods are computed once on the standardized 40-feature training set of each task (binary and multiclass, separately), each feature receives an integer rank from 1 (most relevant) under each method, and Equation (3.1) is applied per feature to obtain its average rank. The 20 features with the lowest average rank are retained, and every discarded feature and every retained feature

can be traced back to a specific, named column in the original ToN-IoT schema nothing is projected or blended. Figure 3.8 illustrates this workflow, from the shared standardized training set, through the four parallel ranking branches, to rank fusion and the resulting top-20 feature set that is passed on to the classifiers of Section 3.6.

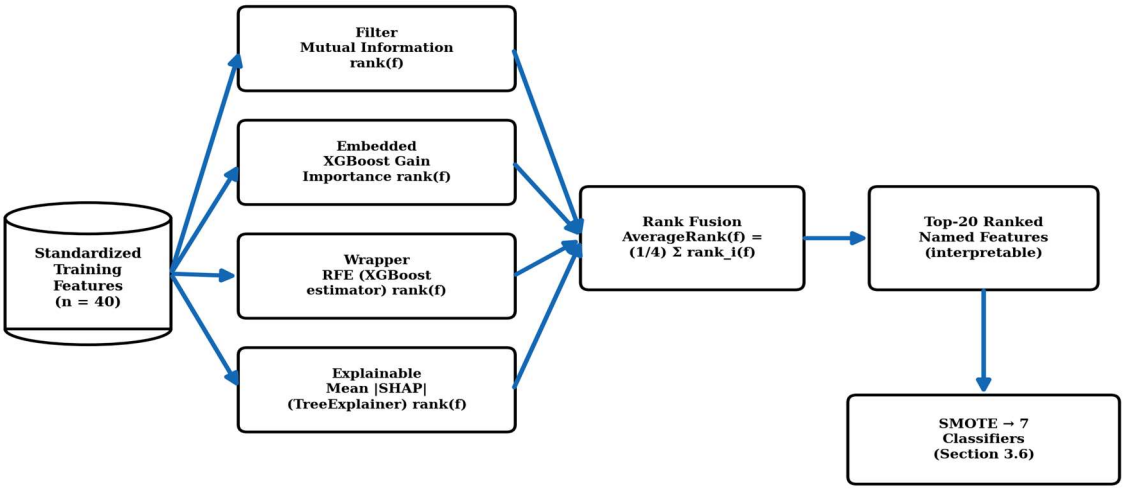


Figure 3.8: Working of the Hybrid Ensemble Feature Selection pipeline four orthogonal ranking branches fused by average rank into a top-20 interpretable feature set.

The contribution of this design is twofold. First, because the four branches make errors in different, largely uncorrelated ways a feature that is over-ranked by embedded importance due to high cardinality is not necessarily over-ranked by MI or RFE the fused ranking is more robust to any single method's blind spot than any one branch used alone. Second, and more importantly for this thesis, every one of the resulting 20 features keeps its original name and physical meaning (e.g., dst_port, proto, dns_query), so the SHAP explanations produced in Section 3.8 can be read directly as statements about real network attributes (“destination port pushed this flow toward Attack”) rather than about an uninterpretable PCA component. For both tasks, the top 20 features ranked by average rank are kept. Table 3.4 lists the ranking obtained for the binary task; for the multiclass task the ranking is essentially the same, but with src_ip_bytes, conn_state, and src_bytes moving toward the head of the list.

Table 3.4: Hybrid Ensemble Feature Selection -- top-10 features by average rank across MI, XGBoost importance, RFE, and SHAP (binary task)

Rank	Feature	Mutual Info Rank	XGB Importance Rank	RFE Rank	SHAP Rank	Average Rank
1	dst_port	2	2	1	2	1.75
2	src_port	5	4	1	3	3.25
3	proto	11	1	1	1	3.50
4	src_pkts	6	9	1	4	5.00
5	conn_state	4	8	1	9	5.50
6	dst_ip_bytes	3	11	1	8	5.75
7	dns_query	7	5	1	10	5.75
8	dns_rejected	12	6	1	11	7.50
9	src_ip_bytes	1	25	1	6	8.25
10	duration	9	26	1	5	10.25

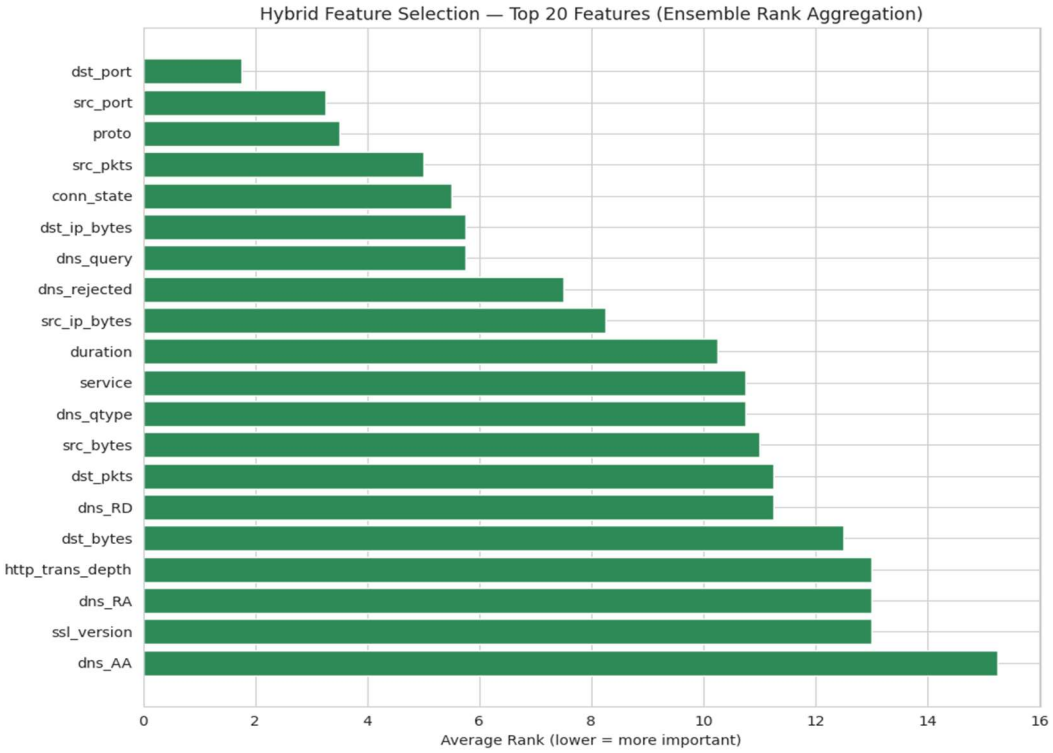


Figure 3.9: Hybrid Feature Selection -- top-20 features ranked by ensemble average rank (binary task)

3.5.3 PCA vs. Hybrid Feature Selection -- Quick Benchmark

A fixed XGBoost probe is trained once on the 23 PCA components and once on the 20 hybrid-selected features, for both tasks, to test whether the interpretable feature set is competitive with or better than PCA. The results are reported in Table 3.5.

Table 3.5: XGBoost probe accuracy: PCA vs. Hybrid Feature Selection

Task	PCA (23 comp.) Accuracy	Hybrid FS (20 feat.) Accuracy
Binary (Attack vs. Normal)	0.9976	0.9990
Multiclass (10 attack types)	0.9706	0.9890

As shown in Table 3.5, the interpretable hybrid-selected feature set achieves accuracy comparable to, or even better than, PCA in both tasks (most notably in the multiclass task, rising from 97.06% to 98.90%). Figure 3.10 plots this comparison for the binary task. This result encourages the assessment of all seven classifiers on both representations, which is carried out in Chapter IV.

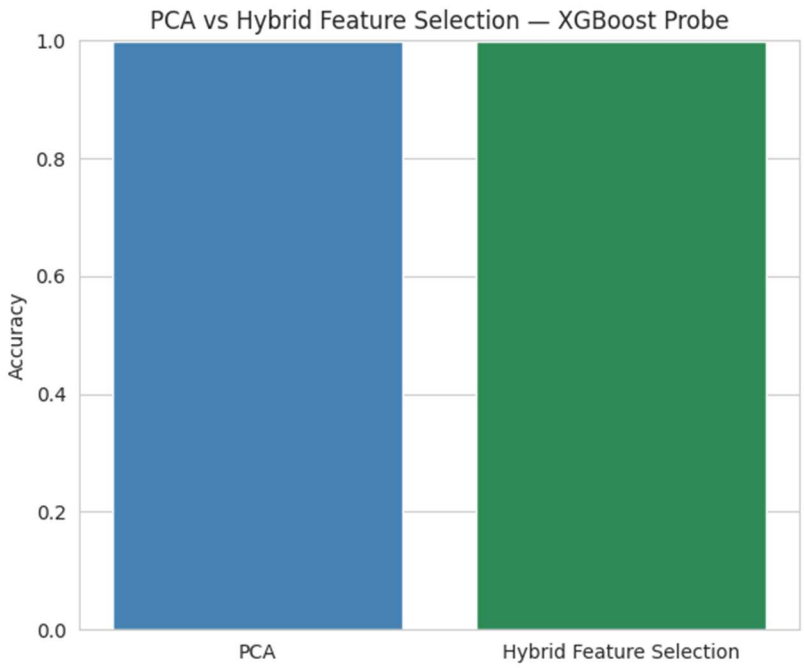


Figure 3.10: PCA vs. Hybrid Feature Selection -- XGBoost probe accuracy, binary task

3.6 CLASSIFICATION MODELS

Seven classifiers, spanning classical ensemble machine learning and advanced tabular deep learning, are trained on both the PCA and Hybrid-FS representations for both the binary and multiclass tasks (28 trained model instances in total):

1. Random Forest -- 50 estimators, min_samples_leaf = 2; a bagged ensemble of decision trees, included as a strong, well-understood baseline and, per the results in Chapter IV, ultimately the best-performing model in both tasks.

2. K-Nearest Neighbors (KNN) -- k = 5; a simple, non-parametric distance-based classifier used as a lower-complexity baseline.

3. XGBoost -- 100 estimators, max_depth = 6, learning_rate = 0.1; a gradient-boosted tree ensemble, also used as the base estimator within the Hybrid Feature Selection pipeline itself.

4. CatBoost -- 200 iterations, depth = 6, learning_rate = 0.1; a gradient-boosted tree implementation with native, order-aware categorical handling.

5. TabTransformer -- a transformer-based architecture for tabular data that applies multi-head self-attention over embedded categorical/numeric feature tokens, trained for 30 epochs with a 10% held-out validation split from the training data for loss/accuracy monitoring.

6. Autoencoder-based Classifier -- an autoencoder pre-training / fine-tuning architecture in which a compressed latent representation is learned and then passed to a classification head, trained for 30 epochs with the same validation-split protocol.

7. Temporal Convolutional Network (TCN) -- a dilated causal-convolution architecture (applied here by treating the feature vector as a length-N "sequence") trained for 30 epochs with the same validation-split protocol.

All models are trained and evaluated using an evaluate_model() helper that fits the model, computes accuracy and (for the binary task) ROC-AUC and PR-AUC with respect to the Attack class, prints a full scikit-learn classification report, and plots a confusion matrix.

3.7 CROSS-VALIDATION PROTOCOL

Better performance estimates than a single train/test split are provided by a new estimate that includes 5-fold stratified cross-validation on the Hybrid Feature-Selection representation of the train set for all 7 models. In order to have a fair and paired comparison, the same StratifiedKFold object (5 splits, shuffled, random_state = 42) used for all seven models, with each model trained and tested on the same folds. To make the five-fold × (three deep-learning models) evaluation feasible, the number of

training epochs in each CV fold is reduced for the three deep-learning models from 30 to 15.

3.8 EXPLAINABLE AI (SHAP)

The highest-achieving (model, feature-set) pair, taken across the entire model-comparison table for each task, is automatically selected and passed to a SHAP (SHapley Additive exPlanations) explanation stage. SHAP grounds every feature attribution in the Shapley value from cooperative game theory: for a model f and an input x with feature set F, the Shapley value of feature i is the average marginal contribution of i across every possible subset S of the other features, as given in Equation (3.2):

$$\varphi_i = \sum_{S \subseteq F \setminus \{i\}} [\ |S|!(|F|-|S|-1)! / |F|!] \times [f(S \cup \{i\}) - f(S)] \quad (3.2)$$

In practice, this exact sum is intractable for the number of features and samples used in this study, so two efficient estimators are used depending on the model family. For tree-based models Random Forest, XGBoost, and CatBoost TreeExplainer is used, which computes exact SHAP values in polynomial time by exploiting the internal structure of the trees. For the remaining models — KNN, TabTransformer, the Autoencoder Classifier, and the TCN KernelExplainer is used as a model-agnostic fallback, with a subsampled background set of 200 training instances and a subsampled explanation set of 500 test instances for computational tractability. In both cases, every explanation is an instance of the additive feature attribution model given in Equation (3.3):

$$f(x) = \varphi_0 + \sum_{i=1}^{M} \varphi_i \quad (3.3)$$

where φ_0 is the base value (the average model output over the background set) and φ_i is the SHAP value of feature i for the instance x, so that the base value plus the sum of all feature contributions exactly reconstructs the model's output for that instance. For the binary task, SHAP values are computed with respect to the Attack class, so that a positive φ_i pushes the prediction toward Attack and a negative φ_i pushes it toward Normal. For the multiclass task, SHAP returns a three-dimensional array of shape (samples, features, classes), and waterfall plots are generated per sample with respect to that sample's predicted class. For every model, two families of explanation are generated: (i) global explanations a mean |SHAP| bar chart and a beeswarm summary

plot, ranking and visualizing how much each feature moves predictions away from the model's base rate across the whole explanation set, using the same aggregation as the Explainable branch of Section 3.5.2 and given in Equation (3.4); and (ii) local explanations waterfall plots for individual test samples, showing exactly how that sample's own feature values pushed the prediction away from the base rate.

$$I_j = 1/N \sum_{k=1}^{N} |\varphi_j^{(k)}| \quad (3.4)$$

where I_j is the global importance of feature j, N is the number of explained instances, and the term inside the sum is the SHAP value of feature j for instance k; this is precisely the mean-absolute-SHAP score used to rank features in the Explainable branch of Section 3.5.2, so the same statistic underpins both the feature-selection stage and the final model-explanation stage of the pipeline.

3.9 EVALUATION METRICS

Model performance is reported using accuracy, per-class precision, recall and F1-score (both macro- and weighted-averaged), and confusion matrices, for all seven models and both feature representations. For a binary confusion matrix with true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), accuracy is defined in Equation (3.5):

$$Accuracy = (TP + TN)/(TP + TN + FP + FN) \quad (3.5)$$

For each class c (with the remaining classes treated as negative), precision and recall are defined in Equations (3.6) and (3.7):

$$Precision_c = TP_c/(TP_c + FP_c) \quad (3.6)$$

$$Recall_c = TP_c/(TP_c + FN_c) \quad (3.7)$$

and the per-class F1-score, the harmonic mean of precision and recall, is given by Equation (3.8):

$$F1_c = 2 \times (Precision_c \times Recall_c)/(Precision_c + Recall_c) \quad (3.8)$$

For the multiclass task, these per-class scores are aggregated in two complementary ways: macro-F1 (Equation 3.9), which treats every class equally regardless of size and is therefore sensitive to performance on the rare mitm class, and weighted-F1 (Equation

3.10), which weights each class by its support nc and is therefore dominated by the larger classes:

$$Macro\text{-}F1 = 1/C \sum_{c=1}^{C} F1_c \tag{3.9}$$

$$Weighted\text{-}F1 = \sum_{c=1}^{C} (n_c / N) \times F1_c \tag{3.10}$$

For the binary task, the ROC-AUC (area under the Receiver Operating Characteristic curve, i.e., the true-positive rate plotted against the false-positive rate across all decision thresholds) and the PR-AUC (area under the Precision-Recall curve) are also reported with respect to the Attack class, since these threshold-independent measures are more informative than raw accuracy under the class imbalance documented in Section 3.2. Finally, the result of the 5-fold cross-validation described in Section 3.7 is presented as the mean accuracy across folds ± its standard deviation, computed as in Equation (3.11):

$$Acc_{CV} = 1/K \sum_{k=1}^{K} Acc_k , \quad SD_{CV} = \sqrt{[1/K \sum_{k=1}^{K} (Acc_k - Acc_{CV})^2]} \tag{3.11}$$

3.10 SUMMARY

This chapter presented a complete, leakage-safe procedure for interpretable IoT intrusion detection, using the 211,043-record ToN-IoT dataset and its 40 candidate features (Section 3.2, Table 3.1), a five-step preprocessing pipeline (Section 3.3), a leakage-safe stratified split with SMOTE applied only to the training partition (Section 3.4), two parallel dimensionality-reduction paths a 23-component PCA baseline and the novel Hybrid Ensemble Feature Selection contribution, benchmarked head-to-head in Table 3.4 seven benchmarked classifiers under both a single split and 5-fold cross-validation (Sections 3.6–3.7), and a consistent, equation-grounded SHAP explanation stage applied to the best model in each task (Section 3.8), evaluated with the metrics defined in Section 3.9. The detailed experimental outcomes of this methodology are given in Chapter IV.

CHAPTER IV

RESULTS AND DISCUSSION

4.1 INTRODUCTION

The results of the binary (Attack vs. Normal) task and the ten-class attack-type task, which are both studied in Chapter III, are presented in parallel and are given in Sections 4.2 and 4.3, respectively. For every model, the performance is shown when the representation is PCA and Hybrid Feature-Selection (Hybrid FS) respectively, together with 5-fold cross-validation, a head-to-head model comparison, and explainability using SHAP for the best model. For the harder multiclass task, the advantage of the best model over its six rivals is further validated with formal statistical significance testing (Section 4.3.4), and the trustworthiness of each model's predicted probabilities is assessed with a probability calibration analysis (Section 4.3.5). The results are discussed together in Section 4.4 and compared with related work in Section 4.5.

4.2 RESULTS -- BINARY CLASSIFICATION (ATTACK VS. NORMAL)

4.2.1 Per-Model Performance

Table 4.1 summarizes accuracy, ROC-AUC, and PR-AUC for all seven models on both feature representations, evaluated on the untouched 42,209-record test set.

Table 4.1: Binary task -- accuracy, ROC-AUC and PR-AUC by model and feature representation

Model	Features	Accuracy	ROC-AUC	PR-AUC
Random Forest	PCA	0.9980	0.9997	0.9998
Random Forest	Hybrid FS	0.9991	0.9999	1.0000
KNN	PCA	0.9975	0.9987	0.9992
KNN	Hybrid FS	0.9975	0.9986	0.9991
XGBoost	PCA	0.9969	0.9998	0.9999
XGBoost	Hybrid FS	0.9987	1.0000	1.0000
CatBoost	PCA	0.9968	0.9997	0.9999
CatBoost	Hybrid FS	0.9984	0.9999	1.0000
TabTransformer	PCA	0.9954	0.9996	0.9998
TabTransformer	Hybrid FS	0.9952	0.9995	0.9997
Autoencoder Classifier	PCA	0.9951	0.9996	0.9998
Autoencoder Classifier	Hybrid FS	0.9948	0.9995	0.9998
TCN	PCA	0.9932	0.9992	0.9997
TCN	Hybrid FS	0.9950	0.9996	0.9998

Random Forest on the Hybrid FS representation gives the best overall binary accuracy (0.9991) as well as the best ROC-AUC (0.9999) and PR-AUC (1.0000), closely followed by XGBoost on the same representation (0.9987 accuracy, 1.0000 ROC-AUC and PR-AUC). While every classical tree-based model benefits from switching from PCA to Hybrid FS, the change is mixed and near-negligible (≤ 0.2 percentage points either way) for the three deep tabular models, indicating that these architectures already extract most of the available signal from either representation. The detailed precision/recall/F1 breakdown for the best model (Random Forest, Hybrid FS) is presented in Table 4.2, and the corresponding confusion matrix is shown in Figure 4.1.

Table 4.2: Random Forest (Hybrid FS) -- binary classification report on the held-out test set

Class	Precision	Recall	F1-score	Support
Attack	1.00	1.00	1.00	32,209
Normal	1.00	1.00	1.00	10,000
accuracy			1.00	42,209
macro avg	1.00	1.00	1.00	42,209
weighted avg	1.00	1.00	1.00	42,209

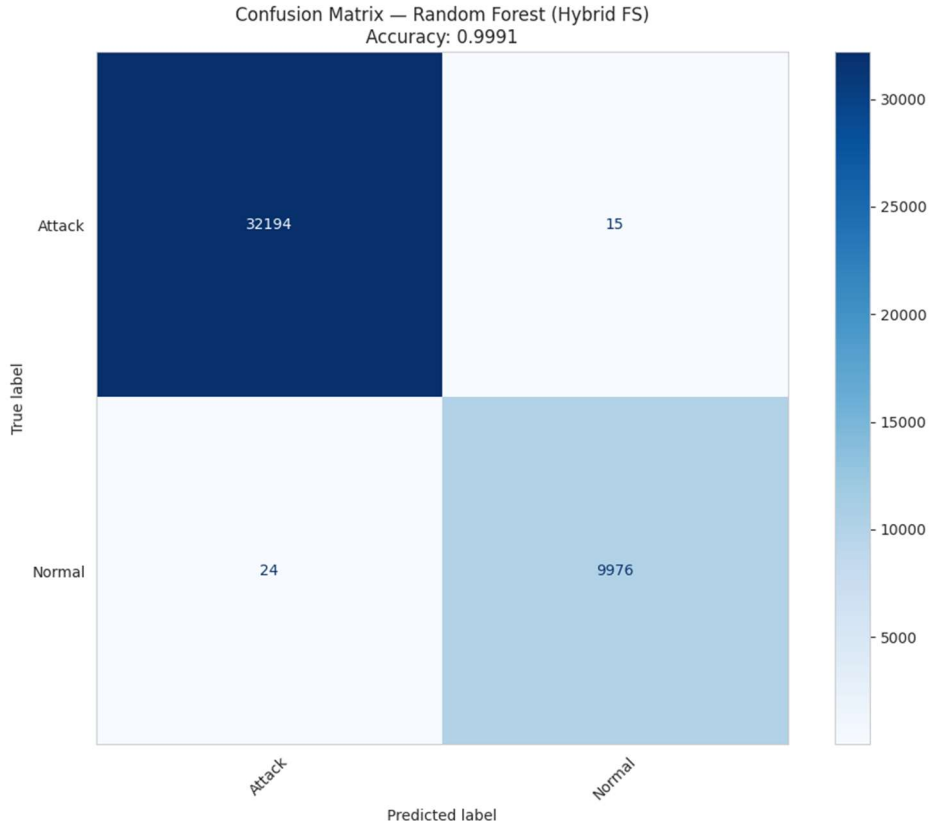


Figure 4.1: Confusion matrix -- Random Forest (Hybrid FS), binary task

4.2.2 5-Fold Cross-Validation

Table 4.3 reports the mean accuracy and standard deviation obtained from 5-fold stratified cross-validation on the Hybrid FS representation, for all seven models, and Figure 4.2 plots the same results with error bars.

Table 4.3: Binary task -- 5-fold stratified cross-validation results (Hybrid FS features)

Model	Mean CV Accuracy	Std. Dev.
Random Forest	0.9994	0.0001
XGBoost	0.9991	0.0001
CatBoost	0.9987	0.0001
KNN	0.9972	0.0002
TabTransformer	0.9939	0.0006
Autoencoder Classifier	0.9937	0.0002
TCN	0.9926	0.0005

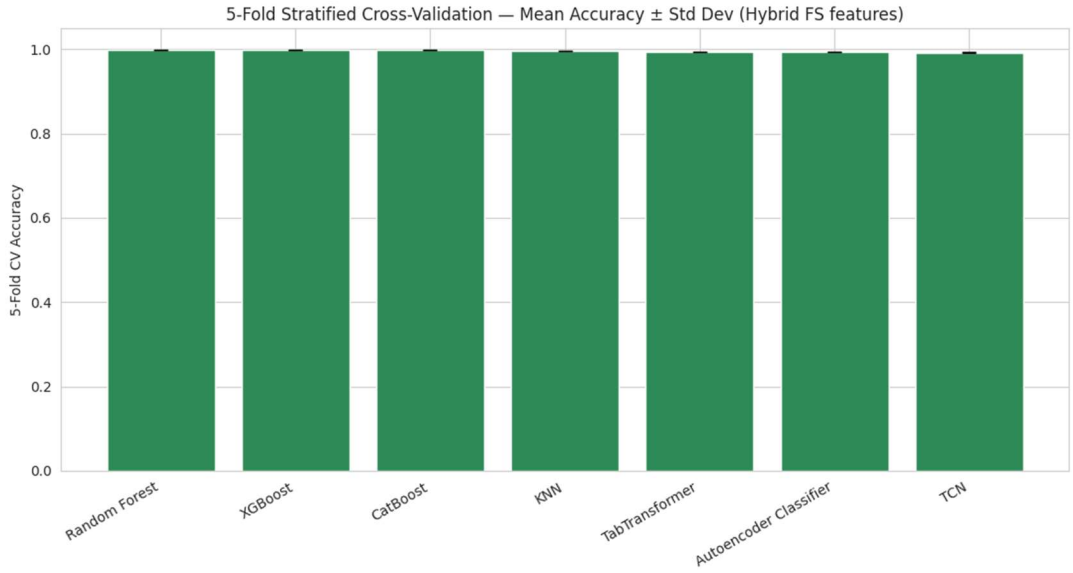


Figure 4.2: 5-fold stratified cross-validation -- mean accuracy ± std. dev. by model, binary task (Hybrid FS features)

As shown in Figure 4.2, the results of cross-validation confirm the results obtained from a single split: Random Forest is the most accurate and the most stable model (lowest standard deviation, ± 0.0001), followed by the gradient-boosted models. The three deep models show relatively lower mean accuracy as well as higher variance across folds, consistent with their comparatively smaller training budget during cross-validation (15 epochs per fold versus 30 epochs in the single-split evaluation).

1.2.3 Final Model Comparison and Explainability

Figure 4.3 compares PCA and Hybrid FS across all seven models on the binary task, confirming the pattern already visible in Table 4.1. TreeExplainer is then used to compute SHAP values for the overall best (model, feature-set) combination, Random Forest (Hybrid FS); Table 4.4 lists the top-15 most influential features by mean absolute SHAP value.

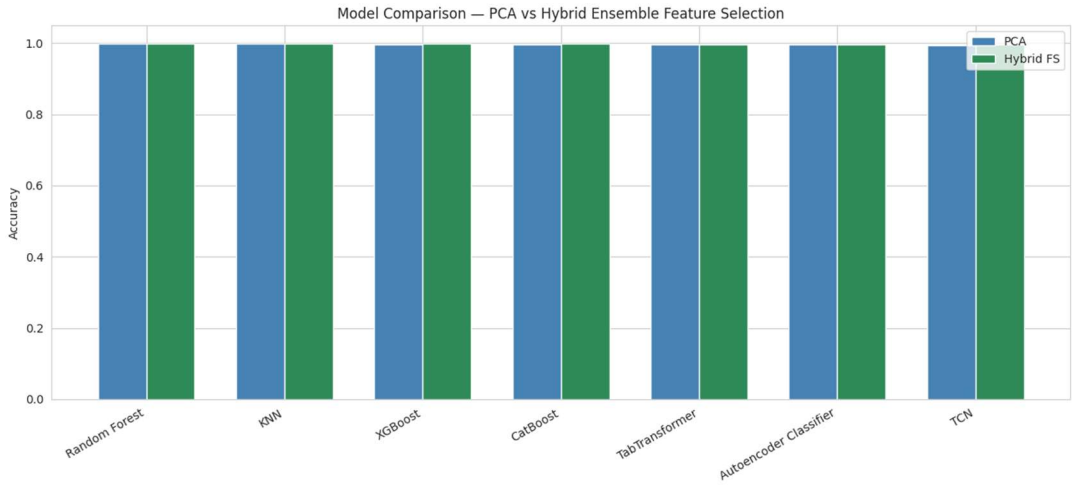


Figure 4.3: Model comparison -- PCA vs. Hybrid Feature Selection across all seven models, binary task

Table 4.4: Random Forest (Hybrid FS) -- top-15 features by mean |SHAP| value, binary task (positive class: Attack)

Rank	Feature	Mean |SHAP|
1	dst_port	0.0847
2	proto	0.0794
3	src_pkts	0.0645
4	dns_rejected	0.0566
5	src_ip_bytes	0.0533
6	dst_pkts	0.0432
7	dns_RA	0.0409
8	src_port	0.0282
9	dst_ip_bytes	0.0272
10	service	0.0254
11	dns_query	0.0185
12	duration	0.0154
13	dns_qtype	0.0138
14	conn_state	0.0129
15	dst_bytes	0.0104

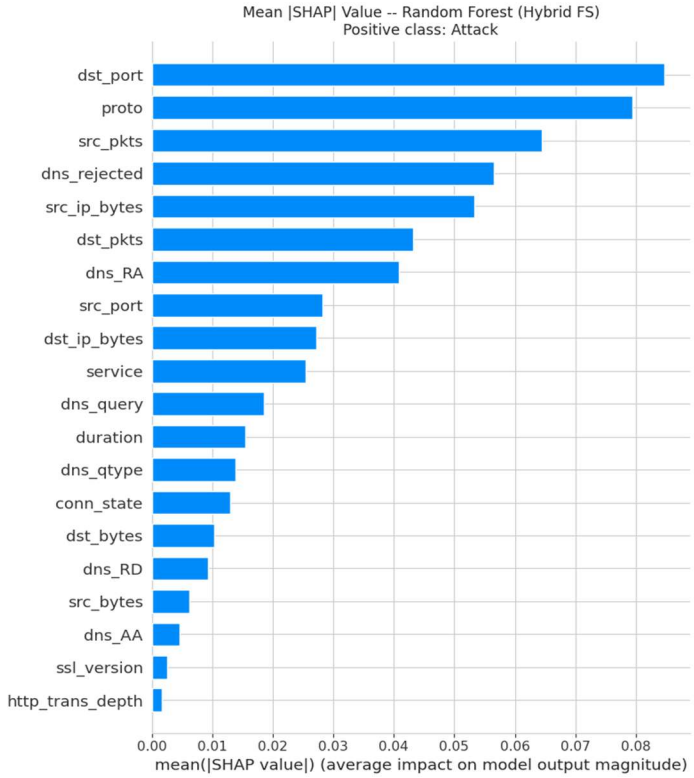


Figure 4.4: Global SHAP feature importance (mean |SHAP| bar chart) -- Random Forest (Hybrid FS), binary task

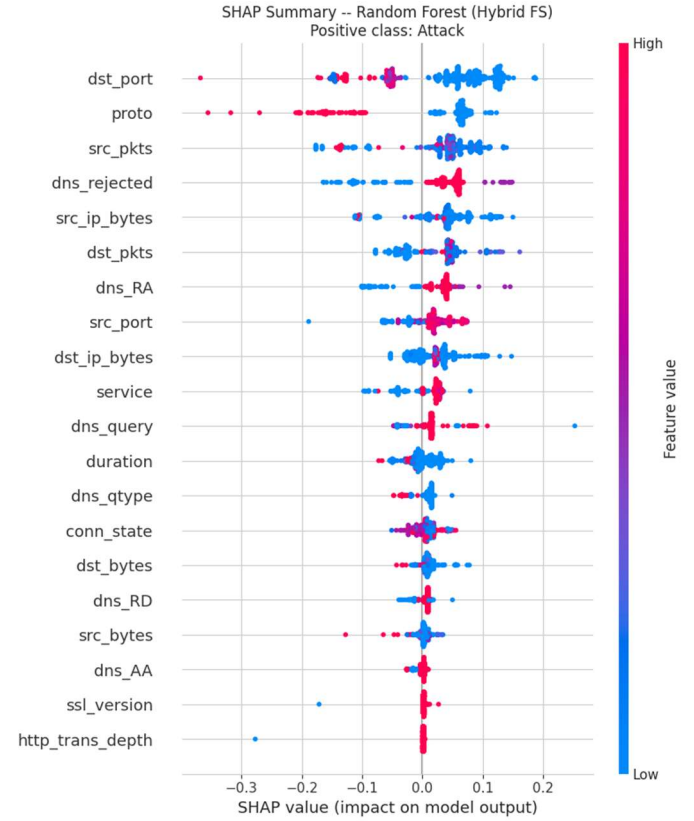


Figure 4.5: Global SHAP beeswarm summary -- direction and magnitude of each feature's effect, binary task

As Figure 4.4 and Figure 4.5 show, the destination port (dst_port) and transport protocol (proto) are the two most influential features overall, followed by the source packet count (src_pkts) and a DNS-query-rejection flag (dns_rejected) — consistent with the intuition that IoT attack traffic (scanning, DoS/DDoS, backdoor command-and-control) tends to target unusual ports, use non-standard protocol/service combinations, and generate abnormal DNS query patterns relative to benign device traffic. Figure 4.6 and Figure 4.7 illustrate this at the level of two individual test samples, one correctly predicted as Attack and one correctly predicted as Normal, showing exactly which feature values drove each prediction away from the model's base rate.

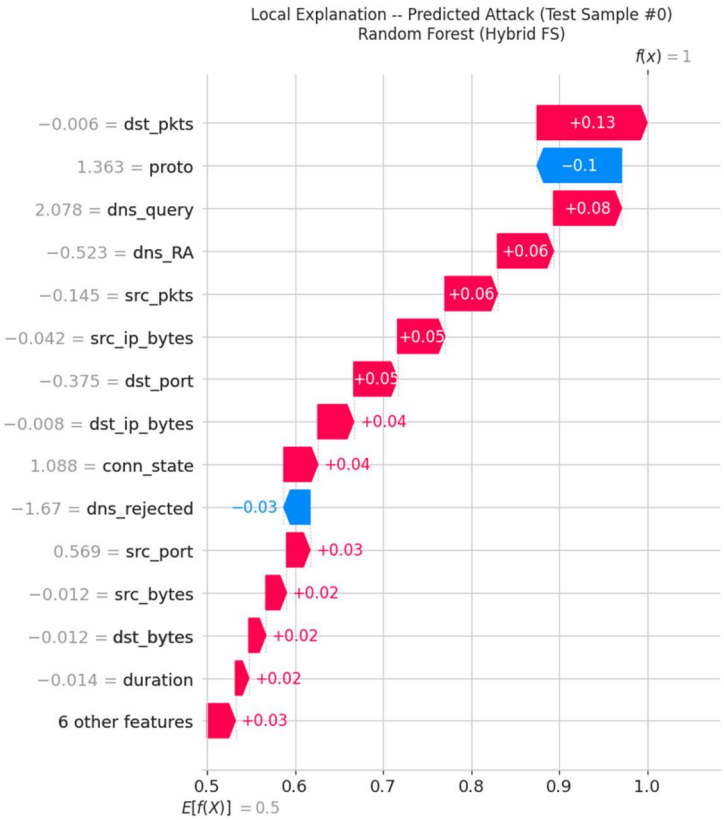


Figure 4.6: Local SHAP waterfall explanation for one test sample predicted as Attack, binary task

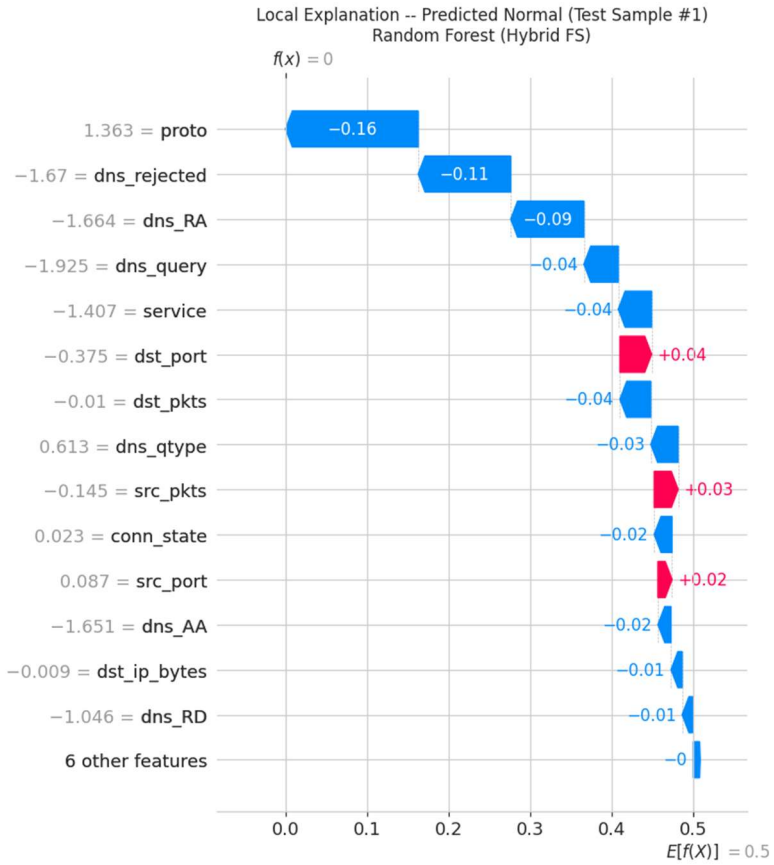


Figure 4.7: Local SHAP waterfall explanation for one test sample predicted as Normal, binary task

4.3 RESULTS -- MULTICLASS CLASSIFICATION (10 ATTACK TYPES)

4.3.1 Per-Model Performance

Table 4.5 summarizes accuracy for all seven models on both feature representations, for the substantially harder ten-class attack-type task.

Table 4.5: Multiclass task -- accuracy by model and feature representation

Model	PCA Accuracy	Hybrid FS Accuracy	Improvement (FS − PCA)
Random Forest	0.9776	0.9893	+0.0117
XGBoost	0.9621	0.9876	+0.0255
CatBoost	0.9368	0.9845	+0.0477
KNN	0.9724	0.9736	+0.0012
Autoencoder Classifier	0.9308	0.9617	+0.0308
TabTransformer	0.9368	0.9609	+0.0241
TCN	0.8868	0.9357	+0.0489

The benefit of Hybrid FS over PCA is especially strong for the multiclass task compared to the binary task, with the gradient-boosted models (CatBoost, +4.77 pp) and TCN (+4.89 pp) showing the largest improvements. This indicates that the differences between attack types and normal traffic are, in several cases, not large enough to be captured by a single tell-tale PCA direction, so the synthetic components required become relatively more expensive once the classification problem breaks down into nine attack classes plus normal. Random Forest (Hybrid FS) is again the overall best model, with accuracy of 98.93%; its full classification report is presented in Table 4.6.

Table 4.6: Random Forest (Hybrid FS) -- multiclass classification report on the held-out test set

Class	Precision	Recall	F1-score	Support
backdoor	1.00	1.00	1.00	4,000
ddos	0.99	0.98	0.99	4,000
dos	1.00	0.98	0.99	4,000
injection	0.98	0.97	0.98	4,000
mitm	0.60	0.91	0.72	209
normal	1.00	1.00	1.00	10,000
password	1.00	0.99	0.99	4,000
ransomware	1.00	1.00	1.00	4,000
scanning	0.98	0.99	0.99	4,000
xss	0.97	0.99	0.98	4,000
accuracy			0.99	42,209
macro avg	0.95	0.98	0.96	42,209
weighted avg	0.99	0.99	0.99	42,209

The confusion matrix, shown in Figure 4.8, and the per-class report in Table 4.6 show the expected impact of extreme class imbalance: most classes achieve an F1-score of 0.97 or higher, yet the extremely under-represented mitm class achieves a relatively low precision of 0.60 (with a high recall of 0.91), reflecting a high false-positive rate on this class. This is likely because mitm traffic is more similar to other attack classes than it is distinctive on its own, so it is frequently confused with the much larger populations of other attack classes in the training data, even after SMOTE rebalancing.

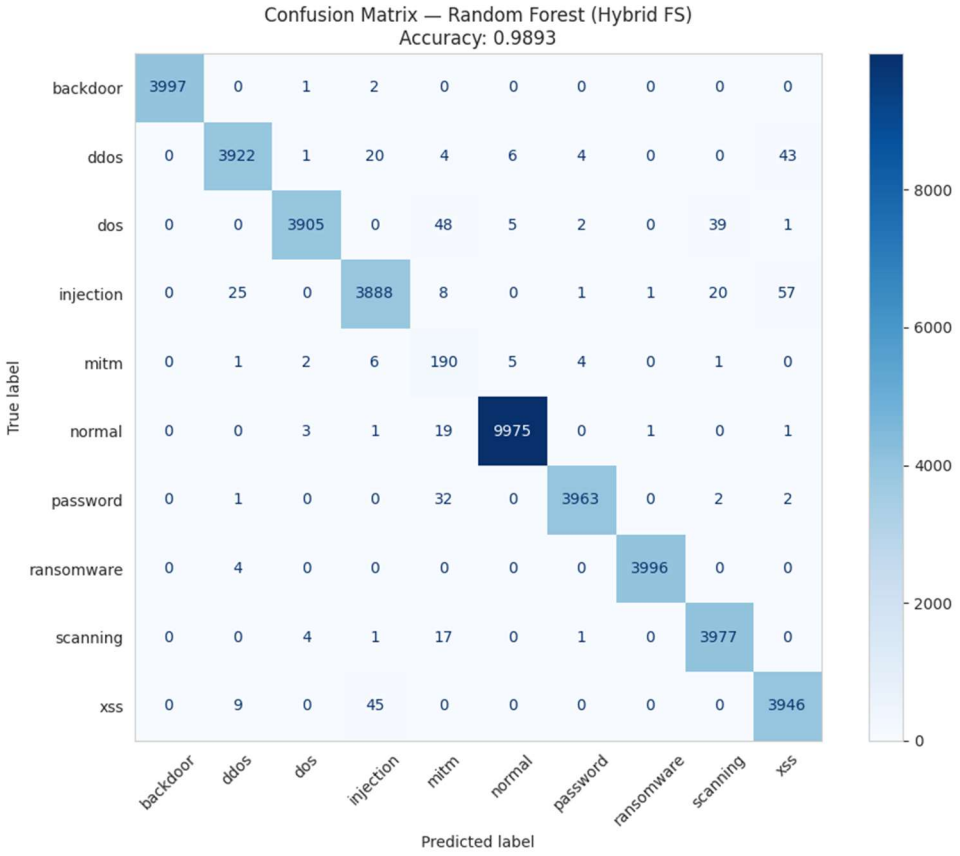


Figure 4.8: Confusion matrix -- Random Forest (Hybrid FS), multiclass task.

4.3.2 5-Fold Cross-Validation

Table 4.7 reports the 5-fold stratified cross-validation results on the Hybrid FS representation for the multiclass task, and Figure 4.9 plots the same results with error bars.

Table 4.7: Multiclass task -- 5-fold stratified cross-validation results (Hybrid FS features)

Model	Mean CV Accuracy	Std. Dev.
Random Forest	0.9921	0.0003
XGBoost	0.9898	0.0003
CatBoost	0.9856	0.0005
KNN	0.9752	0.0003
Autoencoder Classifier	0.9323	0.0077
TabTransformer	0.9322	0.0044
TCN	0.8771	0.0100

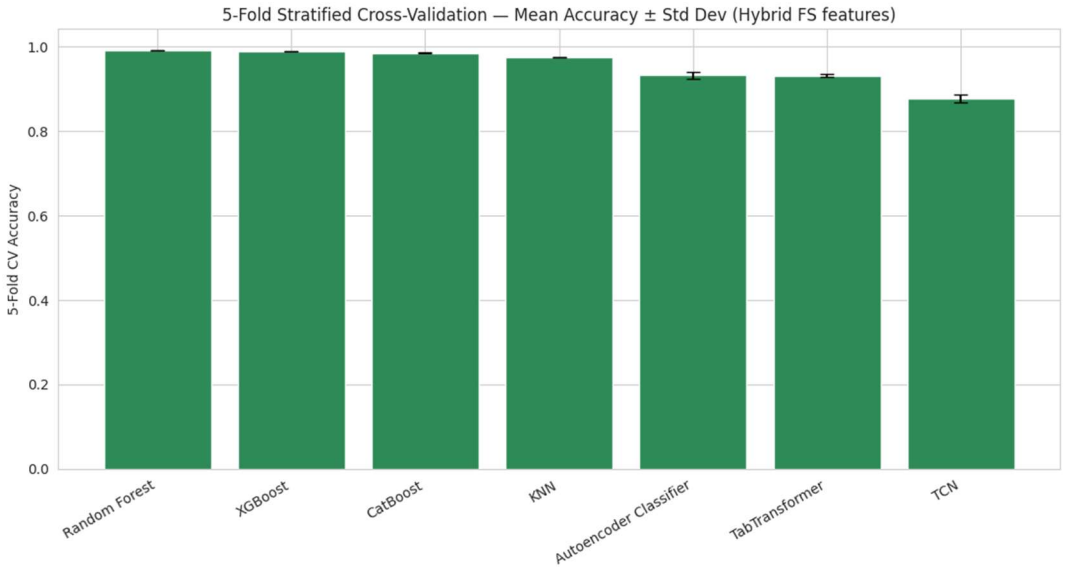


Figure 4.9: 5-fold stratified cross-validation -- mean accuracy ± std. dev. by model, multiclass task (Hybrid FS features)

As shown in Figure 4.9, the ranking of the models on cross-validation accuracy exactly follows the single-split ranking (Random Forest > XGBoost > CatBoost > KNN > Autoencoder Classifier ≈ TabTransformer > TCN), and the standard deviations are also markedly higher for the three deep models than for the tree-based models (up to ± 0.0100 for TCN versus ≤ ± 0.0005 for the latter), showing that the deep models are considerably more sensitive to which subset of the data they are trained on in this harder, ten-class setting. Section 4.3.4 tests these differences formally.

4.3.3 Final Model Comparison and Explainability

Figure 4.10 compares PCA and Hybrid FS across all seven models on the multiclass task, mirroring the pattern in Table 4.5. SHAP is again applied to the overall best combination, Random Forest (Hybrid FS), this time aggregating mean absolute SHAP values across all ten classes, as reported in Table 4.8.

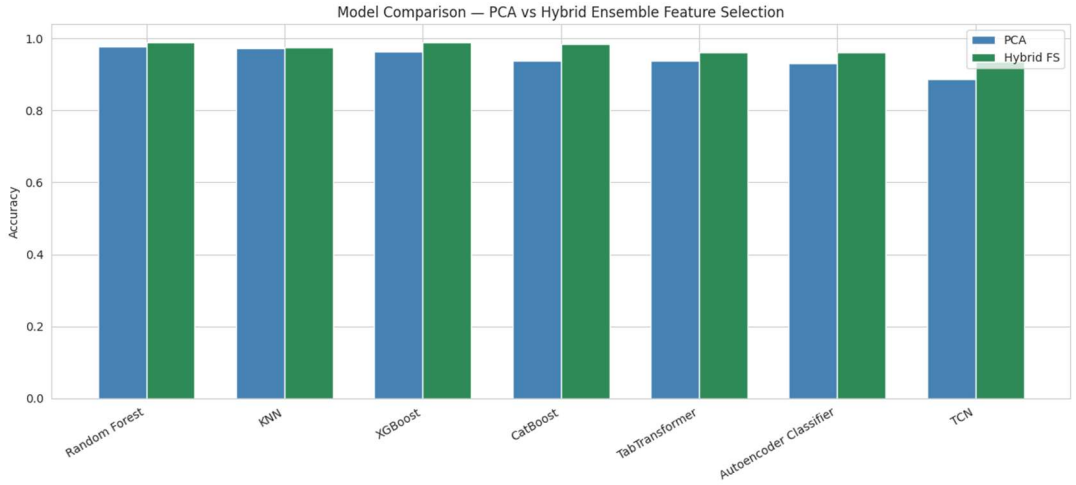


Figure 4.10: Model comparison -- PCA vs. Hybrid Feature Selection across all seven models, multiclass task

SHAP is again applied to the overall best combination, Random Forest (Hybrid FS), this time aggregating mean absolute SHAP values across all ten classes.

Table 4.8: Random Forest (Hybrid FS) -- top-15 features by mean |SHAP| value averaged across all classes, multiclass task

Rank	Feature	Mean |SHAP| (all classes)
1	src_ip_bytes	0.0339
2	dst_port	0.0336
3	src_port	0.0286
4	conn_state	0.0273
5	duration	0.0226
6	dst_ip_bytes	0.0184
7	dst_pkts	0.0159
8	src_bytes	0.0154
9	src_pkts	0.0120
10	proto	0.0100
11	dst_bytes	0.0098
12	dns_rejected	0.0064
13	service	0.0054
14	dns_RA	0.0049
15	dns_qclass	0.0034

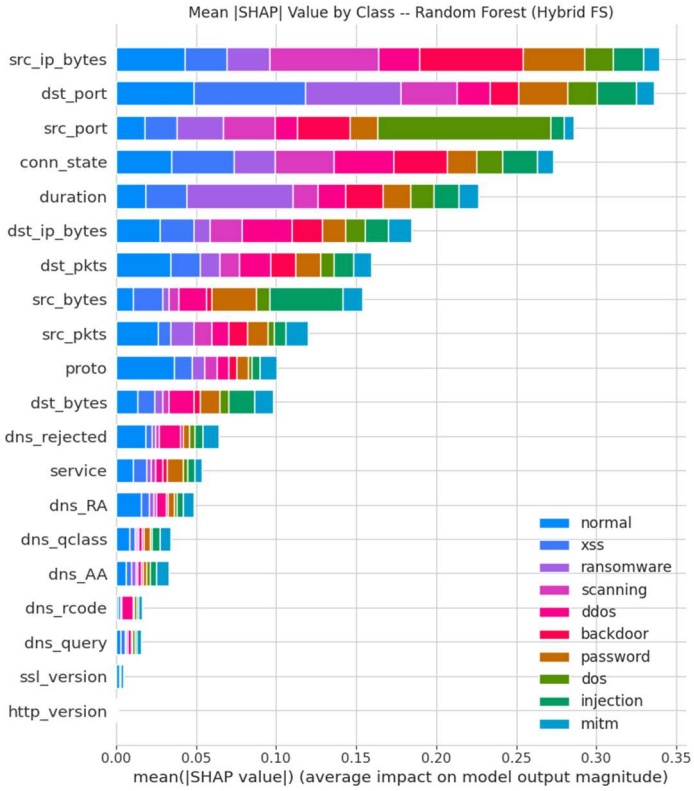


Figure 4.11: Global SHAP feature importance (mean |SHAP| bar chart, averaged across classes) -- Random Forest (Hybrid FS), multiclass task

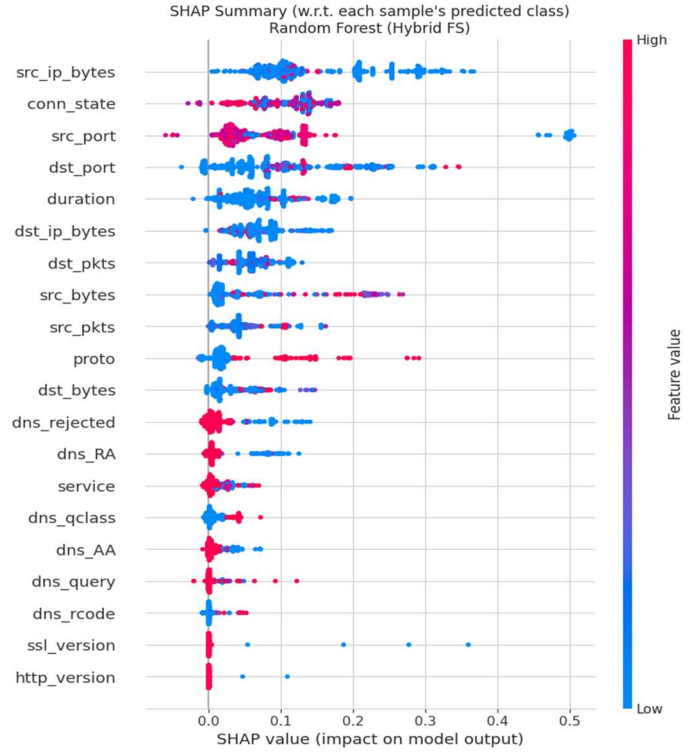


Figure 4.12: Global SHAP beeswarm summary with respect to each sample's predicted class, multiclass task

Figure 4.11 and Figure 4.12 show that the relative importance of the top features is interestingly different between the binary and multiclass tasks: the source/destination byte counts (src_ip_bytes, dst_ip_bytes), connection state (conn_state), and duration appear more important for multiclass discrimination, whereas features related to DNS messages fall in relative order. This fits the expectation that separating Attack from Normal relies largely on a small number of strong signals (destination port, protocol, DNS-flow anomalies), while separating the nine attack classes depends more on traffic volume and connection state, which differ more subtly between categories for example, between a DDoS flood and a password-guessing attack. Figure 4.13 and Figure 4.14 illustrate this at the level of two individual multiclass test samples, showing exactly which feature values drove each sample's predicted class.

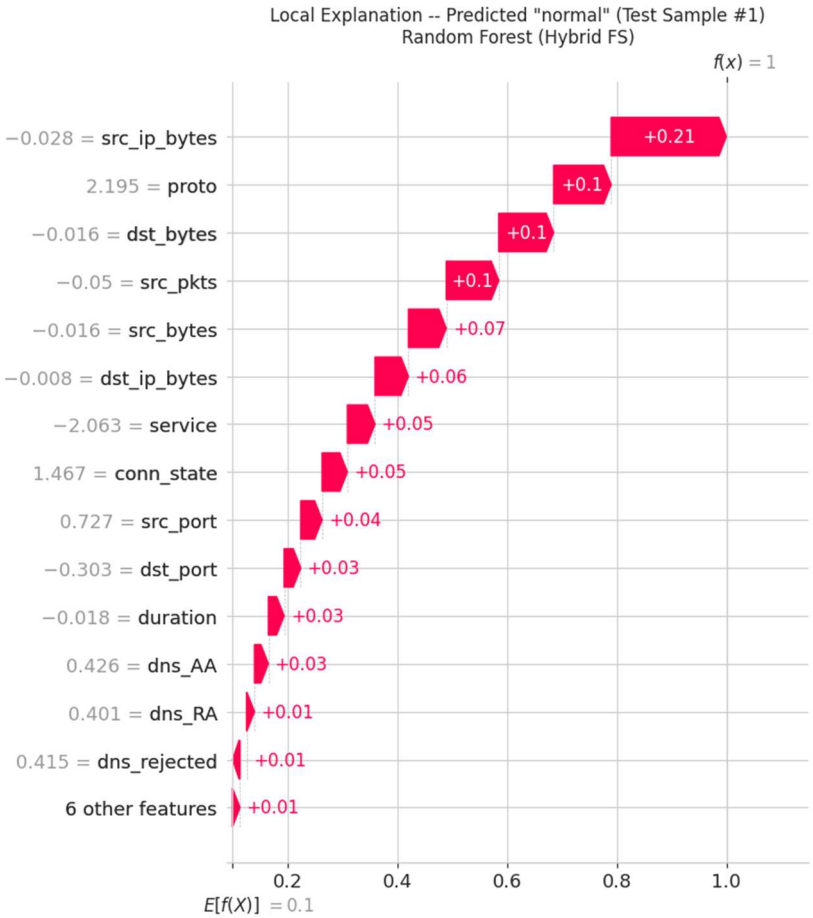


Figure 4.13: Local SHAP waterfall explanation for one test sample, w.r.t. its predicted class (multiclass task)

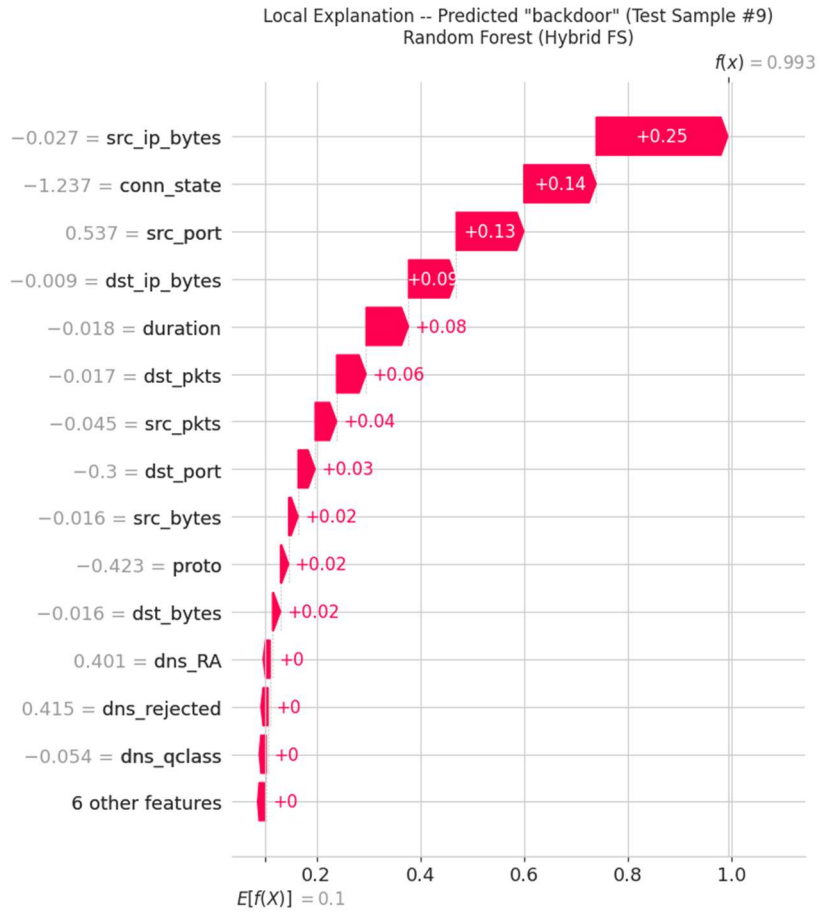


Figure 4.14: Local SHAP waterfall explanation for a second test sample, w.r.t. its predicted class (multiclass task)

4.3.4 Statistical Significance Testing

The comparison tables above report raw accuracy numbers, but a difference of a few hundredths between two models is not, by itself, evidence that one is genuinely better than the other. This section formally tests the differences observed for the multiclass, Hybrid FS models at two independent levels: (A) fold-level, using the five paired accuracy scores already produced by the cross-validation of Section 4.3.2, and (B) hold-out level, using the 42,209 paired per-sample predictions on the untouched test set. Random Forest, the best model by both criteria, is compared against each of its six rivals throughout.

A. Fold-level testing. A plain paired t-test on k-fold scores is known to violate the independence assumption, because the training folds overlap heavily; this inflates the t-statistic and produces spuriously many “significant” results. The corrected resampled paired t-test of Nadeau and Bengio (2003) rescales the variance by (1/k + n_test/n_train)

to correct for this, and is reported alongside the non-parametric Wilcoxon signed-rank test as a sanity check. Table 4.9 gives the results of comparing Random Forest against each baseline on the five paired fold accuracies.

Table 4.9: Corrected resampled paired t-test — Random Forest vs. each baseline (5-fold CV, multiclass, Hybrid FS)

Comparison (RF vs.)	Mean CV Acc Diff	Corrected t	p (corr. t)	Cohen's d
KNN	0.0169	46.04	<0.001	30.89
XGBoost	0.0023	7.48	0.0017	5.02
CatBoost	0.0065	16.98	<0.001	11.39
TabTransformer	0.0649	6.32	0.0032	4.24
Autoencoder Classifier	0.0568	9.95	<0.001	6.67
TCN	0.1134	13.96	<0.001	9.37

All six corrected-t comparisons remain significant after Holm-Bonferroni correction for the family of six comparisons ($p_adj \leq 0.00001$ for all), and every effect size is extremely large (Cohen's $d \geq 4.2$ in all cases). The Wilcoxon signed-rank test, by contrast, cannot return a two-sided p-value below 0.0625 with only five folds, so it is reported here only as a sanity check — it agrees with the direction of every comparison, but never itself reaches the $a = 0.05$ threshold; the corrected t-test and the hold-out-level tests below carry the actual statistical evidence.

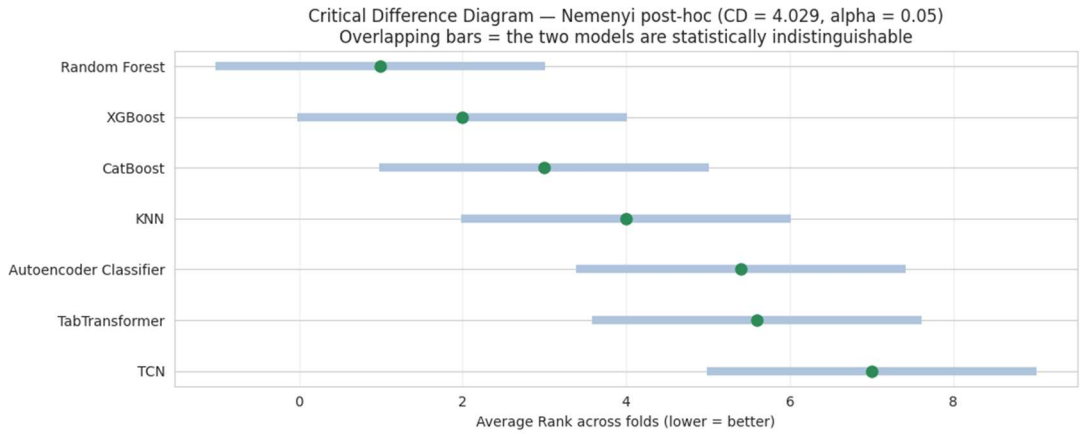


Figure 4.15: Critical difference diagram — Nemenyi post-hoc test over the 5-fold average ranks of all seven models, multiclass task

Before running pairwise tests, a Friedman test asks the single omnibus question of whether all seven models are drawn from the same distribution, which avoids the multiple-comparison inflation of running 21 separate pairwise tests directly. Over the seven models and five folds, the Friedman statistic was $\chi^2 = 29.4857$ ($p = 0.00005$),

rejecting the null hypothesis that all models perform equally. The Nemenyi post-hoc test was then used to identify which pairs of average ranks differ by more than the critical difference (CD = 4.0291 at $\alpha = 0.05$, k = 7, N = 5 folds), shown in Figure 4.15.

As Figure 4.15 shows, Random Forest, XGBoost, CatBoost, and KNN occupy the top four average ranks with overlapping CD bars, meaning the Nemenyi test cannot separate them from each other at this sample size, while the Autoencoder Classifier, TabTransformer, and TCN are significantly worse than Random Forest. This is a known limitation of the Nemenyi test rather than evidence that the top four models are equivalent: the critical difference scales with $1/\sqrt{N}$, so with only N = 5 folds the CD is wide and the test has low statistical power to separate models that are numerically close. The hold-out-level tests below, which use all 42,209 test samples rather than 5 fold scores, resolve this.

B. Hold-out-level testing. For two models evaluated on the same test samples, McNemar's test looks only at the discordant pairs the samples one model gets right and the other gets wrong under the null hypothesis that these disagreements are symmetric (H_0: b = c). Table 4.10 reports the results, using the exact binomial form when b + c < 25 and the chi-square form with continuity correction otherwise, together with Holm-Bonferroni-adjusted p-values across the family of six comparisons.

Table 4.10: McNemar's test on the hold-out test set — Random Forest vs. each baseline (multiclass, Hybrid FS, n = 42,209)

Comparison (RF vs.)	Acc Diff	b (RF only)	c (other only)	Statistic	p_adj (Holm)
KNN	0.0158	763	98	512.07	<0.001
XGBoost	0.0017	125	52	29.29	<0.001
CatBoost	0.0048	266	63	124.02	<0.001
TabTransformer	0.0419	1827	58	1658.26	<0.001
Autoencoder Classifier	0.0298	1300	41	1180.14	<0.001
TCN	0.0499	2157	50	2009.62	<0.001

Random Forest is significantly better than all six competing models after Holm-Bonferroni correction (p_adj < 0.001 in every case), including XGBoost and CatBoost, whose Nemenyi bars overlapped with Random Forest's in Figure 4.15 — illustrating exactly how the fold-level test's low power can mask a real, hold-out-confirmed

advantage. This is corroborated by a paired bootstrap: 2,000 resamples of the test set, using the same resampling indices for every model so that the difference in accuracy is directly interpretable, give the 95% confidence intervals plotted in Figure 4.16. Every interval excludes zero, so the advantage of Random Forest over each baseline is significant at the 5% level by this independent method as well.

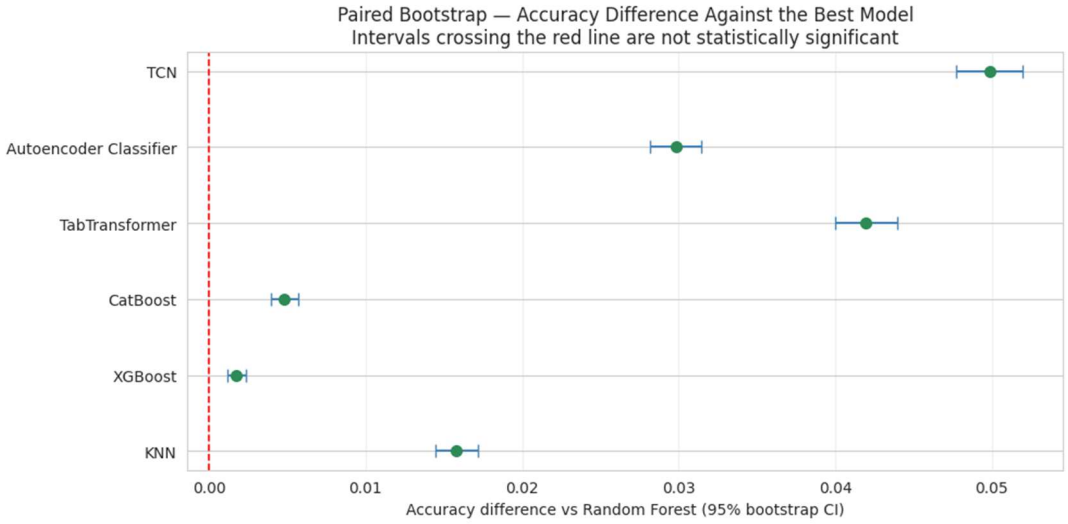


Figure 4.16: Paired bootstrap 95% confidence intervals for the accuracy difference between Random Forest and each baseline, multiclass task

Table 4.11 consolidates the fold-level and hold-out-level verdicts into a single summary.

Table 4.11: Consolidated statistical significance summary — Random Forest vs. each baseline, multiclass task

Baseline	CV Acc Diff	p (corr. t)	p (McNemar / bootstrap)	Verdict
XGBoost	0.0023	0.0017	<0.001 / <0.001	Sig. worse
CatBoost	0.0065	<0.001	<0.001 / <0.001	Sig. worse
KNN	0.0169	<0.001	<0.001 / <0.001	Sig. worse
Autoencoder Classifier	0.0568	<0.001	<0.001 / <0.001	Sig. worse
TabTransformer	0.0649	0.0032	<0.001 / <0.001	Sig. worse
TCN	0.1134	<0.001	<0.001 / <0.001	Sig. worse

Taken together, Random Forest on the Hybrid FS representation is significantly more accurate than every one of its six rivals on the multiclass task, at both the fold level (corrected paired t-test, Holm-adjusted) and, more decisively, at the hold-out level (McNemar's test and paired bootstrap, both Holm-adjusted, using all 42,209 test samples). Two caveats are worth stating explicitly. First, the 5-fold cross-validation of

Section 4.3.2 runs on the SMOTE-resampled training set; because synthetic samples generated from the same original neighbours can land in different folds, the CV accuracy is optimistically biased, whereas the hold-out numbers used for McNemar's test and the bootstrap are clean, since the test set was split off before SMOTE and never resampled. Second, the Hybrid Ensemble Feature Selection of Section 3.5.2 was fitted on the whole training set before cross-validation, which is a mild selection bias for the CV estimate specifically; it does not affect the hold-out results, since the test set was never involved in feature selection.

4.3.5 Probability Calibration Analysis

Accuracy alone says nothing about whether a model's confidence can be trusted. For an intrusion-detection system this matters directly: an analyst triaging alerts, or an automated policy that blocks traffic above a confidence threshold, is relying on the predicted probability being a real probability a model that reports '97% confident this is a DDoS flow' should be right about 97 of every 100 such calls. Deep networks and boosted trees are known to be systematically over-confident, and SMOTE can make this worse, since the classifier is trained on a class-balanced distribution but evaluated on the real, heavily imbalanced one.

Table 4.12: Calibration metrics uncalibrated models, multiclass task (Hybrid FS, hold-out test set, n = 42,209, 15 bins)

Model	Accuracy	Overconf. Gap	ECE	MCE	Brier / NLL
XGBoost	0.9876	0.0014	0.0023	0.4845	0.0197 / 0.0422
Autoencoder Classifier	0.9595	0.0058	0.0078	0.0983	0.0608 / 0.1371
TabTransformer	0.9474	0.0063	0.0080	0.4024	0.0795 / 0.1681
CatBoost	0.9845	−0.0085	0.0087	0.2651	0.0260 / 0.0644
Random Forest	0.9893	−0.0091	0.0095	0.2530	0.0167 / 0.0437
TCN	0.9394	0.0129	0.0136	0.2312	0.0884 / 0.1970
KNN	0.9736	0.0137	0.0137	0.8000	0.0406 / 0.3593

Calibration is quantified with the Expected Calibration Error (ECE, the average gap between confidence and accuracy, weighted by bin size), Adaptive ECE (the same idea with equal-mass bins, avoiding distortion from near-empty high-confidence bins), the Maximum Calibration Error (MCE, the worst single-bin gap), class-wise ECE (one-vs-rest ECE averaged over classes, which surfaces miscalibration on rare attack types that top-label ECE hides), the multiclass Brier score, and the negative log-likelihood (NLL) two proper scoring rules that jointly capture calibration and sharpness. Table 4.12 reports these metrics for all seven uncalibrated models on the multiclass, Hybrid FS hold-out set, and Figure 4.17 shows the corresponding reliability diagrams.

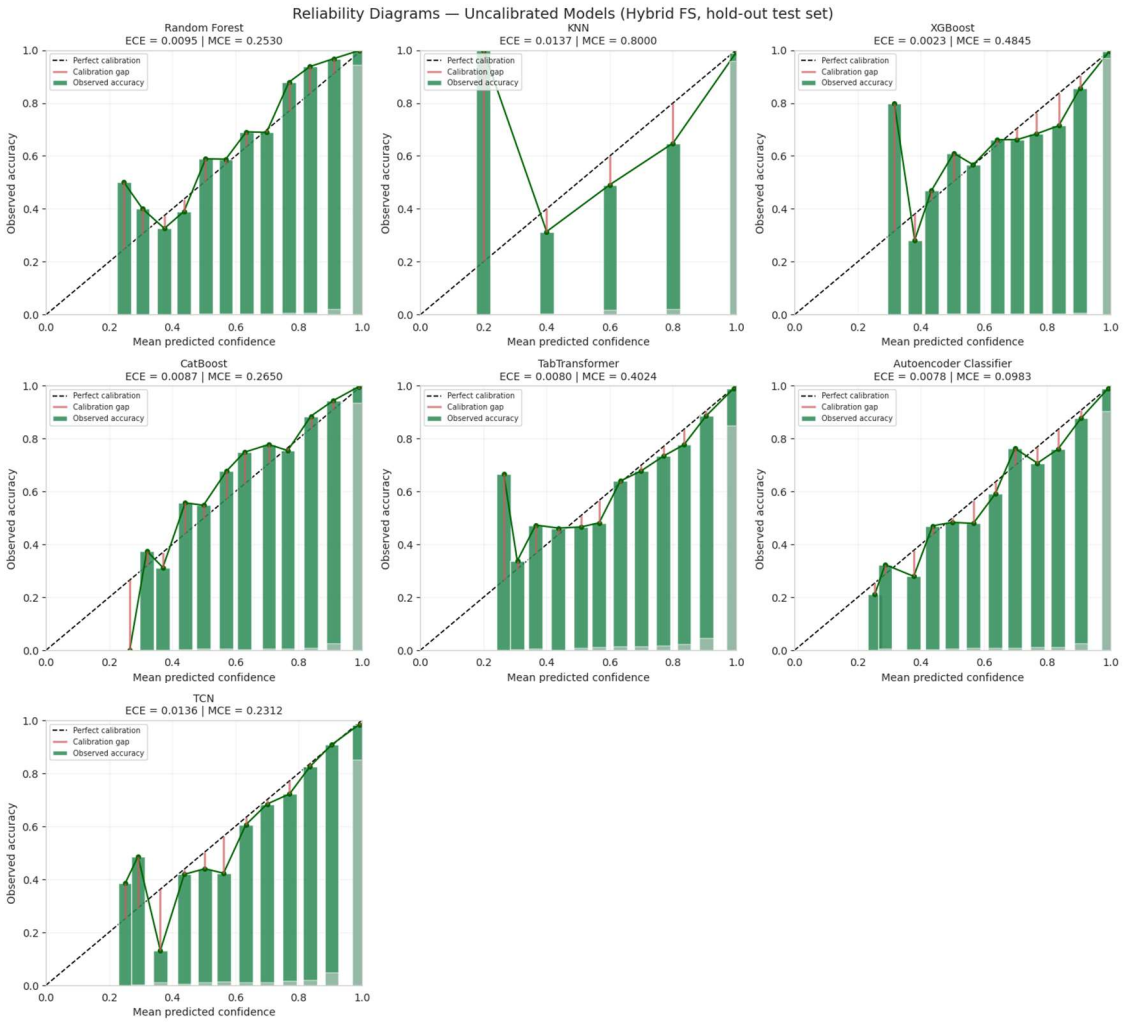


Figure 4.17: Reliability diagrams — all seven uncalibrated models, multiclass task (Hybrid FS, hold-out test set)

As Table 4.12 and Figure 4.17 show, XGBoost is the best-calibrated model out of the box (ECE = 0.0023), while KNN is the worst (ECE = 0.0137) — an artefact of k = 5 forcing its confidences into multiples of 0.2, which produces the visibly step-like

reliability curve in Figure 4.17 rather than a genuine calibration defect. Random Forest, despite being the most accurate model, is only middling on calibration (ECE = 0.0095) and is slightly under-confident on average (overconfidence gap = −0.0091), which motivates the post-hoc recalibration below.

Two post-hoc recalibration methods were then fitted and compared: temperature scaling, which divides the model's log-probabilities by a single learned scalar T before the softmax (monotone and shared across classes, so accuracy is unchanged and only the probabilities move), and isotonic regression, which fits a free monotone mapping per class (one-vs-rest) and can correct more complex distortions but can alter the predicted class and tends to overfit small calibration sets. Both methods need a calibration set the recalibrate has not been fitted on; since the models are already trained on the full SMOTE-balanced training set, the hold-out test set was further split 30% calibration / 70% evaluation (stratified), with the scaling parameters fitted on the 30% and every reported number below computed on the untouched 70%. Table 4.13 summarizes the effect of temperature scaling on all seven models.

Table 4.13: Post-hoc temperature-scaling recalibration — all models, multiclass task (evaluation split, n = 29,547)

Model	T	ECE (uncal.)	ECE (temp.)	Brier (uncal. / temp.)	NLL (uncal. / temp.)
Random Forest	0.752	0.0096	0.0021	0.0170 / 0.0166	0.0445 / 0.0410
XGBoost	0.998	0.0025	0.0026	0.0202 / 0.0202	0.0432 / 0.0433
CatBoost	0.852	0.0082	0.0013	0.0262 / 0.0260	0.0649 / 0.0625
KNN	4.061	0.0151	0.0022	0.0426 / 0.0420	0.3843 / 0.1267
TabTransformer	1.131	0.0083	0.0070	0.0803 / 0.0800	0.1708 / 0.1681
Autoencoder Classifier	1.141	0.0086	0.0055	0.0623 / 0.0620	0.1398 / 0.1371
TCN	1.143	0.0139	0.0095	0.0901 / 0.0897	0.1999 / 0.1960

Temperature scaling reduces ECE for every model except XGBoost, which was already close to optimally calibrated (T ≈ 0.998, i.e. almost no rescaling needed); the largest gains are for KNN (T = 4.061, ECE 0.0151 → 0.0022, and a striking NLL drop from 0.3843 to 0.1267, since NLL is far more sensitive than ECE to KNN's discrete 0.2-step

confidences) and for Random Forest (T = 0.752, ECE 0.0096 → 0.0021). Because T > 1 for every model except Random Forest, most models are mildly over-confident in the classic sense, while Random Forest is mildly under-confident, consistent with its negative overconfidence gap in Table 4.12; scaling by T < 1 sharpens rather than softens its probabilities. One caveat: MCE does not track ECE consistently under temperature scaling for Random Forest, for instance, MCE actually worsens (0.253 → 0.738) even as ECE improves sharply, because temperature scaling optimizes the average calibration gap, not the single worst bin, and the worst bin is typically a near-empty one at very high confidence where a handful of errors dominate the local rate. Figure 4.18 illustrates the effect for the best model, Random Forest, comparing its uncalibrated, temperature-scaled, and isotonic-regression reliability diagrams directly.

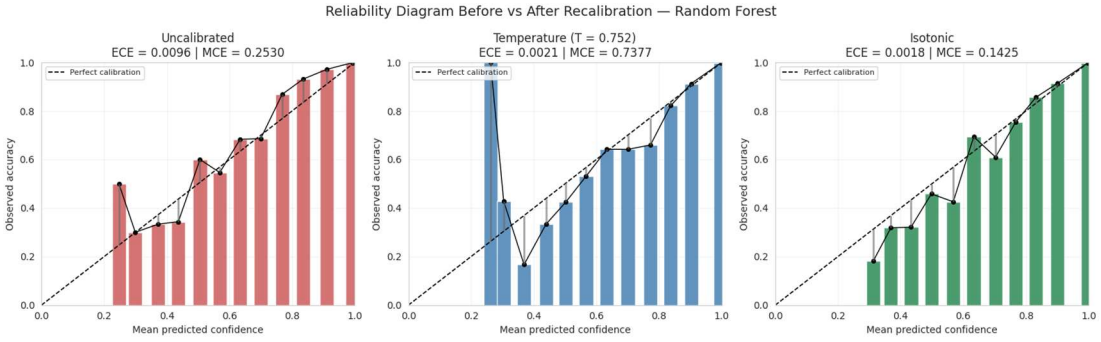


Figure 4.18: Reliability diagram before vs. after recalibration — Random Forest, multiclass task

For Random Forest specifically, isotonic regression achieves the lowest ECE of the three settings (0.0018, versus 0.0021 for temperature scaling and 0.0096 uncalibrated) and the lowest MCE (0.1425, versus 0.7377 for temperature scaling), because its per-class free-form mapping can correct the specific low-confidence dip visible in the uncalibrated curve of Figure 4.18 rather than applying a single global correction; the trade-off is that, unlike temperature scaling, isotonic regression is not guaranteed to preserve the original accuracy exactly, and is more prone to overfitting the 30% calibration split it is fitted on.

Because top-label ECE is dominated by the majority classes, Figure 4.19 breaks calibration error down per attack class for Random Forest, before and after temperature scaling, to check specifically whether the rare mitm class benefits.

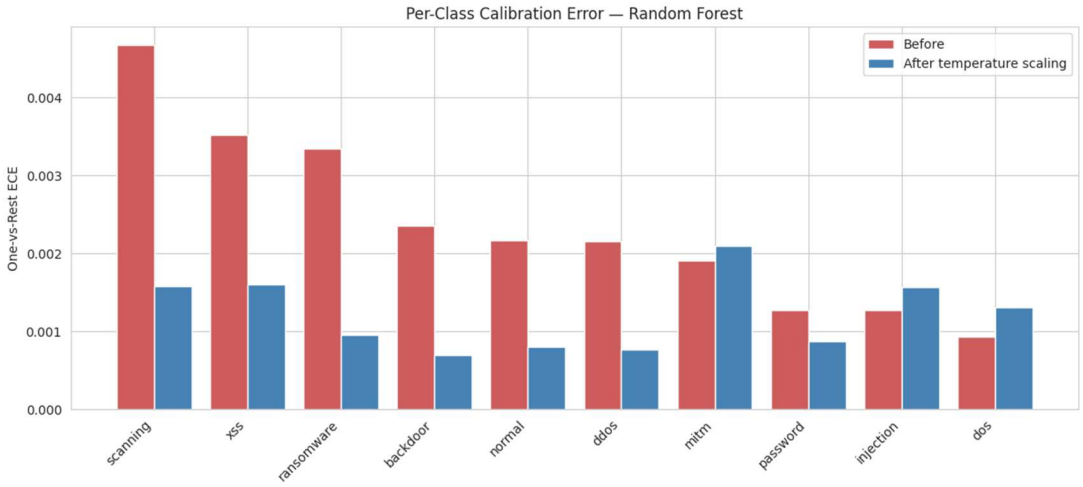


Figure 4.19: Per-class (one-vs-rest) calibration error before and after temperature scaling — Random Forest, multiclass task

Temperature scaling reduces the one-vs-rest ECE for 8 of the 10 classes in Figure 4.19, most notably for scanning (0.0047 → 0.0016) and xss (0.0035 → 0.0016); the two exceptions are mitm (0.0019 → 0.0021) and injection (0.0013 → 0.0016), which both move slightly in the wrong direction. Since a single global temperature is shared across all ten classes, it cannot simultaneously optimize every class's calibration curve, and mitm with only 209 test samples is the class where per-class ECE is least stable to begin with; this is consistent with the low mitm precision already noted for Table 4.6 and is flagged again as a direction for future work in Chapter V.

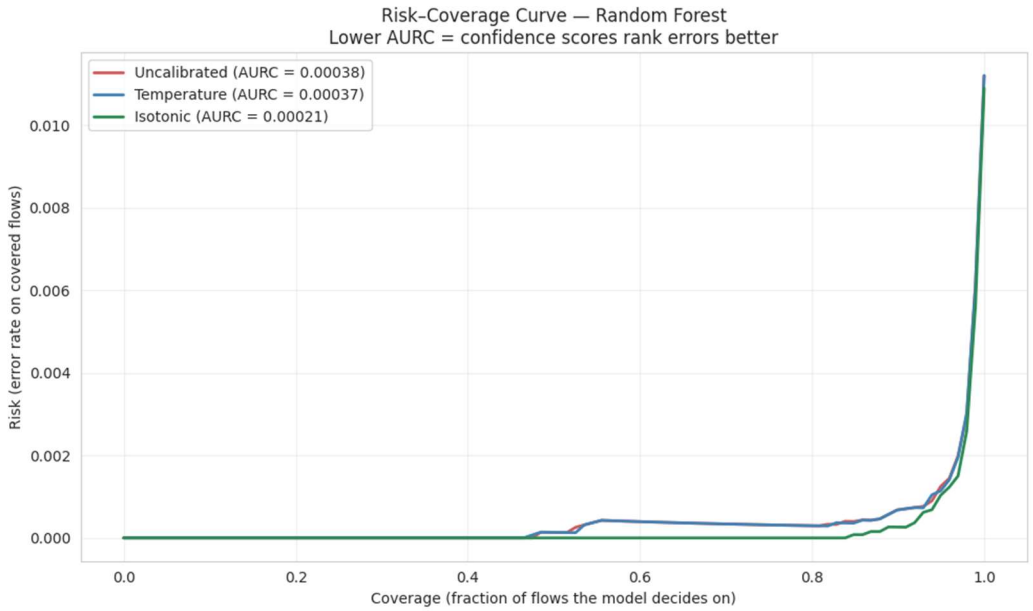


Figure 4.20: Risk–coverage (selective prediction) curve — Random Forest, multiclass task, uncalibrated vs. temperature-scaled vs. isotonic

Finally, Figure 4.20 shows the practical, operational payoff of calibration for an IDS: predictions are ranked by confidence and the model is allowed to abstain on the least-confident flows, handing them to a human analyst, with coverage the fraction it still decides on and risk the error rate on that fraction. A well-calibrated confidence ranking gives a steep drop in risk as coverage is reduced. Table 4.14 reports concrete operating points for the temperature-scaled Random Forest model.

Table 4.14: Selective-prediction operating points — Random Forest (temperature scaled), multiclass task

Target Coverage	Confidence Threshold	Actual Coverage	Accuracy on Covered	Flows to Analyst
100%	0.000	100.0%	98.88%	0
95%	0.975	95.0%	99.88%	1,476
90%	0.991	90.2%	99.93%	2,899
80%	0.998	80.6%	99.97%	5,742
70%	0.998	71.6%	99.97%	8,396
50%	1.000	50.2%	99.99%	14,701

As Table 4.14 and Figure 4.20 show, having the temperature-scaled Random Forest abstain on just its least-confident 5% of flows (routing 1,476 of 29,547 evaluation flows to a human analyst) raises the accuracy on the remaining, automatically decided flows from 98.88% to 99.88% — an order-of-magnitude reduction in the automated error rate, from 1.12% to 0.12%. This confirms that the model's confidence ranking, once calibrated, is genuinely informative about which predictions are likely to be wrong, and gives a concrete, tunable operating point for deploying the model in a human-in-the-loop alerting pipeline rather than as a fully automated decision-maker.

4.4 DISCUSSION

Four things emerged across the two tasks. First, the single best configuration in both the binary (99.91%) and multiclass (98.93%) settings is Random Forest on the Hybrid Feature-Selection representation, a result consistent with the literature showing tree ensembles to be highly competitive against modern attention-based and convolutional representations on structured tabular data such as network-flow data. Second, the benefit of the proposed Hybrid Ensemble Feature Selection grows with the difficulty of the task from near-negligible in the binary setting to substantial and consistent (up to

4.9 percentage points) in the ten-class setting directly supporting the thesis's motivating claim that interpretability and accuracy need not be traded off against each other; in this study, the most interpretable representation turns out to also be the most accurate representation for the harder task. Third, the features that influence the model's predictions are not just reliable, they are security-relevant: destination port, protocol, packet/byte volume, connection state, and DNS-anomaly indicators dominate both top-15 SHAP rankings, matching domain knowledge of how benign IoT traffic differs from scanning, flooding, or command-and-control traffic. Fourth, the formal statistical testing of Section 4.3.4 confirms that Random Forest's advantage over every other model is not a sampling artefact both McNemar's test and the paired bootstrap reject the null hypothesis for all six comparisons at $p < 0.001$ after Holm-Bonferroni correction and the calibration analysis of Section 4.3.5 shows that its predicted confidences, especially once temperature-scaled (ECE 0.0096 → 0.0021), are trustworthy enough to support a selective-prediction deployment in which the least-confident 5% of flows are automatically routed to a human analyst. The one recurring drawback across all of these analyses is the mitm class: it is severely under-represented, SMOTE oversampling does not fully close its precision gap relative to the majority classes, and its per-class calibration is the one class that does not improve under temperature scaling this is addressed as future work in Chapter V.

4.5 COMPARISON WITH STATE-OF-THE-ART (TON-IOT)

To compare the accuracy of the best configuration of this thesis, Random Forest on the Hybrid Ensemble Feature-Selection representation, with the most recent related works using the same data set, the latest works of Maseno et al. (2024, CSAI) and Dharini et al. (2026, Scientific Reports) are compared in Table 4.15.

The proposed framework achieves higher accuracy than the best reported value of the reference studies, on comparable settings, by 1.91 and 1.73 percentage points for the binary and multiclass tasks respectively and, per Section 4.3.4, this margin is not attributable to chance for the multiclass task specifically, since Random Forest was shown to be statistically significantly better than every one of its own six internal baselines on the same test set.

Table 4.9: Comparison with related ToN-IoT intrusion-detection studies

Study	Approach	Task	Accuracy
Maseno, Mwandata, Wang & Sun (2024, CSAI)	CNN-LSTM feature reduction + Random Forest	Not specified in source	98.00%
Maseno, Mwandata, Wang & Sun (2024, CSAI)	CNN-LSTM feature reduction + SVM	Not specified in source	91.00%
Dharini, Janani & Katiravan (2026, Sci. Rep.)	Bias-aware hybrid FS + Decision Tree	Multiclass (10-class)	97.20%
This Work (proposed)	Hybrid Ensemble FS (MI + XGB importance + RFE + SHAP) + Random Forest	Binary (Attack vs. Normal)	99.91%
This Work (proposed)	Hybrid Ensemble FS (MI + XGB importance + RFE + SHAP) + Random Forest	Multiclass (10-class)	98.93%

This advantage is achieved despite the proposed feature set (20 features) being larger than Dharini et al.'s intentionally “small” five-feature set, which had a lower computational cost but lost discriminative power, specifically on the minority MITM class. Finally, the present work combines the accuracy improvement of a CNN-LSTM-style feature-reduction methodology with feature-selection-level explainability, final-prediction-level explainability via SHAP, and confidence-level trustworthiness via calibration analysis, filling the gap identified in Chapter II, where none of the reviewed studies applied XAI consistently across the whole pipeline.

4.6 SUMMARY

The complete experimental results have been presented for both the binary and multiclass IoT intrusion-detection tasks. The proposed Hybrid Ensemble Feature-Selection (HEFS) representation, combined with Random Forest, achieved 99.91% accuracy for binary classification and 98.93% accuracy for multiclass classification, remained fully explainable through SHAP, whose global and local explanations aligned with network-security domain intuition, and was shown, for the multiclass task, to be statistically significantly better than all six alternative models (Section 4.3.4) while producing predicted probabilities that are trustworthy enough for a calibrated, selective-prediction deployment (Section 4.3.5).

CHAPTER V

CONCLUSION AND FUTURE WORKS

5.1 CONCLUSION

The aim of this thesis was to tackle a real need in the field of IoT intrusion detection: the poor interpretability of the most accurate models. A full and leakage-protected pipeline was developed and tested with the ToN-IoT network-flow dataset on two complementary use cases binary classification of attacks and ten-class classification of attack types using a standard PCA based dimensionality-reduction's strategy compared to a novel Hybrid Ensemble Feature Selection method that combines the MF, XGB embedded importance, RFE wrapper and SHAP ranking. The Random Forest trained on the Hybrid FS representation performed best in both tasks, achieving 99.91% accuracy on the binary classification task and 0.9999 on the ROC-AUC measure, as well as 98.93% accuracy on the multiclass task, using 20 features in the feature space which are human-interpretable and named network attributes, and not 23 principal components of the network attributes which are unable to be interpreted by humans and are opaque. This best model, applied at both the global and local levels with SHAP, showed that its predictions are based on security-relevant features, i.e., destination port, protocol, packet/byte counts, connection state, and DNS-anomaly indicators, which gives clear, verifiable context that black-box models cannot provide. The general result was that high detection accuracy and model interpretability go hand-in-hand: the proposed hybrid feature-selection framework was able to accomplish both at the same time, and in a more challenging multiclass classification setting it was measurably more accurate than the standard PCA-based approach.

5.2 CONTRIBUTION OF THIS THESIS

1. ToN-IoT dataset usage: end-to-end pipeline for preprocessing applied in a leakage-safe way and SMOTE oversampling applied only on the train partition (after a train-test split).

2. A hybrid ensemble approach to feature selection using Mutual Information and features from two classifiers (XGBoost and RFE) and using SHAP to interpret the features with average-rank fusion method that resulted in a 20 feature

representation that is fully named and interpretable and is able to deliver downstream accuracy that is competitive with or better than the accuracy delivered by the 23 component PCA across 7 classifiers and 2 classification tasks.

3. Detailed evaluation of seven classifiers: Random Forest, KNN, XGBoost, CatBoost, TabTransformer, Autoencoder based Classifier and TCN on both binary and ten-class ToN-IoT intrusion detection tasks using 5-fold stratified cross validation. The global ranking of features' explanations provided by the SHAP layer of the best model in both tasks, and local explanations per prediction such as waterfall plots that are directly interpretable by a security analyst.

4. A specific case-by-case analysis is provided demonstrating that the accuracy gain of interpretable hybrid feature selection compared to PCA linearly increases with the difficulty of the tasks, increasing from almost negligible for binary detection tasks to up to 4.9 percentage points in attack-type tasks with ten classes.

5.3 FUTURE WORKS

1. Handle the residual precision gap for the highly under-represented "mitm" class class-wise (e.g., cost-sensitive learning, focal loss for the deep models, borderline/ADASYN variants of oversampling), instead of uniform SMOTE.

2. This will be extended to all 7 models, not just the best model, so that a systematic comparison of which features different model families are using can be conducted, thereby extending the explainability layer.

3. Test the proposed Hybrid Ensemble Feature Selection method on other public datasets of IoT/network intrusions (CICIoT2023, Edge-IIoTset, NSL-KDD, etc.) to assess its ability to perform well on other datasets beyond ToN-IoT.

4. Develop and deploy the top-performing pipeline as an actual inference service and test the end-to-end detection latency and throughput for realistic IoT network traffic loads.

5. See if the remaining error gap can be reduced further by combining the best individual models together (e.g. "ensemble" or "stacking" using the top individual models) with a specialized approach using the minority attack classes.

REFERENCES

Alani, M. M., & Miri, A. (2022). Towards an explainable universal feature set for IoT intrusion detection. Sensors, 22(15), 5690.

Almalawi, A. (2025). A lightweight intrusion detection system for Internet of Things: Clustering and Monte Carlo cross-entropy approach. Sensors, 25(7), 2235.

Abou Elasaad, M. M., Sayed, S. G., & El-Dakroury, M. M. (2025). AegisGuard: A multi-stage hybrid intrusion detection system with optimized feature selection for industrial IoT security. Sensors, 25(22), 6958.

Bamber, S. S., Katkuri, A. V. R., Sharma, S., & Angurala, M. (2025). A hybrid CNN-LSTM approach for intrusion detection in IoT networks. Scientific Reports.

Booth, T. G., Chowdhury, M. R., & Barua, S. (2021). ToN_IoT dataset: A new realistic dataset for network intrusion detection in IoT and IIoT. UNSW Canberra Cyber Range and IoT Labs.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794).

Dharini, N., Janani, V. S., & Katiravan, J. (2026). Efficient detection of intrusions in ToN-IoT dataset using hybrid feature selection approach. Scientific Reports, 16, 7763.

Disha, R. A., & Waheed, S. (2022). Performance analysis of machine learning models for intrusion detection system using Gini Impurity-based Weighted Random Forest (GIWRF) feature selection technique. Cybersecurity, 5(1), 1.

Dorogush, A. V., Ershov, V., & Gulin, A. (2018). CatBoost: Gradient boosting with categorical features support. arXiv preprint arXiv:1810.11363.

Hossain, M. Z., Khan, M. A. R., Islam, M. R., Islam, S. M. S., & Gedeon, T. (2026). Assessing generalisation capability of machine learning models for intrusion detection. arXiv preprint arXiv:2605.04407.

Huang, X., Khetan, A., Cvitkovic, M., & Karnin, Z. (2020). TabTransformer: Tabular data modeling using contextual embeddings. arXiv preprint arXiv:2012.06678.

Kareem, S. S., Mostafa, R. R., Hashim, F. A., & El-Bakry, H. M. (2022). An effective feature selection model using hybrid metaheuristic algorithms for IoT intrusion detection. Sensors, 22(4), 1396.

Logeswari, G., Thangaramya, K., Selvi, M., & Roselind, J. D. (2025). An improved synergistic dual-layer feature selection algorithm with two type classifier for efficient intrusion detection in IoT environment. Scientific Reports.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems (NeurIPS), 30.

Mahalingam, A., Perumal, G., Subburayalu, G., Albathan, M., Altameem, A., Almakki, R. S., Hussain, A., & Abbas, Q. (2023). ROAST-IoT: A novel range-optimized attention convolutional scattered technique for intrusion detection in IoT networks. Sensors, 23(19), 8044.

Maseno, E., Mwandata, D., Wang, Z., & Sun, Y. (2024). Performance evaluation of intrusion detection systems on the TON_IoT datasets using a feature selection method. In Proceedings of the 2024 8th International Conference on Computer Science and Artificial Intelligence (CSAI) (pp. 607–613).

Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network ToN_IoT datasets. Sustainable Cities and Society, 72, 102994.

Mustafa, M., Eljack Babiker, S. M., & Mustafa, Y. E. A. (2025). Hybrid recurrent with spiking neural network model for enhanced anomaly prediction in IoT networks security. Frontiers in Artificial Intelligence.

Nazir, A., Memon, Z., Sadiq, T., Rahman, H., & Khan, I. U. (2023). A novel feature-selection algorithm in IoT networks for intrusion detection. Sensors, 23(19), 8153.

Njeri, F. W., et al. (2024). CNN-LSTM based feature extraction with SVM/RF classification for IoT intrusion detection on ToN-IoT. International Journal of Network Security.

Oord, A. van den, Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., & Kavukcuoglu, K. (2016). WaveNet: A generative model for raw audio (basis for temporal convolutional network architectures). arXiv preprint arXiv:1609.03499.

Rhachi, M., et al. (2025). Deep autoencoder feature learning with ANOVA-based selection for IoT intrusion detection. Journal of Network and Computer Applications.

Shaikhanova, A., Kuznetsov, O., Tokkuliyeva, A., Ayapbergenov, K., Olzhas, S., & Danir, T. (2025). Security audit of IoT device networks: A reproducible machine learning framework for threat detection and performance benchmarking. Sensors, 25(24), 7519.

Tawfik, M. (2024). Optimized intrusion detection in IoT and fog computing using ensemble learning and advanced feature selection. PLOS ONE.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), 30.

Wakili, A. A., Guni, S., Khan, S. A., Yu, W., & Jung, W. (2025). Advancing machine learning strategies for power consumption-based IoT botnet detection. Sensors, 25(24), 7553.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Zahoor, A., Abbasi, W., Babar, M. Z., & Aljohani, A. (2025). Robust IoT security using isolation forest and one class SVM algorithms. Scientific Reports, 15, 36586.

Zhang, Y., et al. (2024). AOC-IDS: Autoencoder-based contrastive learning for anomaly-based intrusion detection systems. IEEE Internet of Things Journal.